SHARDA UNIVERSITY

Plot No. 32-34, Knowledge Park III,
Greater Noida, Uttar Pradesh 201310

Laboratory File

Machine Learning with AWS LAB

CTP303

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING SCIENCE & ENGINEERING

Submitted By	
Student Name	Aditya Sarin
Student Id	2023517099
Section/Group	CS L/01
Programme	B.Tech.
Department	Computer Science and Engineering
Session/Semester	2026-27/07
Submitted To	
Faculty Name	Dr. Neetu Faujdar

TABLE OF CONTENT

S. No.	Name of Experiment	Date of Experiment	Date of Submission	Signature	Remarks
1.	Write a program to implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.	24/07/2026			
2.	Write a program to implement the Candidate Elimination algorithm using AWS services and generate the version space by maintaining Specific (S) and General (G) hypothesis boundaries.	31/07/2026			
3.	Write a program to construct and analyze a Decision Tree classifier using the ID3 algorithm by computing entropy and information gain, and to implement the solution using AWS services.	07/08/2026			
4.	Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions.	14/08/2026			
5.	Write a program to implement a Naïve Bayes Classifier on tabular data using EC2 and Jupyter and evaluate the model accuracy.	21/08/2026			
6.	Write a program to classify a set of documents use the naïve Bayesian Classifier model.	11/09/2026			
7.	Write a program to construct a Bayesian network for the diagnosis of heart patients using standard Heart Disease Data Set.	18/09/2026			
8.	Write a program to apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using k-Means algorithm. Compare the results of these two algorithms.	25/09/2026			
9.	Write a program to implement the non-parametric Locally Weighted Regression algorithm in order to fit data points.	09/10/2026			
10.	Lab 3.1 - Amazon SageMaker - Creating and importing data	23/10/2026			
	Lab 3.2 - Exploring Data				
	Lab 3.3 - Encoding Categorical Data				
	Lab 3.4 - Training a model				
	Lab 3.5 - Generating model performance metrics				
	Lab 3.6 - Hyperparameter Tuning				

EXPERIMENT 1

Aim: To implement the FIND-S algorithm using Amazon EC2 to identify the most specific hypothesis consistent with the given training data.

Apparatus / Software

- AWS Account (Free Tier)
- **Amazon EC2 (Amazon Linux 2)**
- Python 3.x
- Jupyter Notebook
- CSV Dataset (`enjoysport.csv`)

AWS Service Used

- **Amazon EC2** – To execute the FIND-S algorithm and store the dataset locally

Theory

FIND-S is a supervised machine learning algorithm used for **concept learning**.
It determines the **most specific hypothesis** that satisfies all **positive training examples**.

- Starts with the most specific hypothesis
- Generalizes only when necessary
- Ignores negative examples

This experiment demonstrates FIND-S implementation in a **cloud-based EC2 environment**.

Algorithm Steps

1. Initialize hypothesis h with the most specific values (Ø).
2. Read training examples sequentially.
3. For each example:
    - If positive → compare with hypothesis
    - Replace mismatched attributes with ?
    - If negative → ignore
1. Continue until all samples are processed.
2. Output the final hypothesis.

Procedure (Using Only Amazon EC2)

STEP 1: Launch EC2 Instance

1. AWS Console → **EC2**

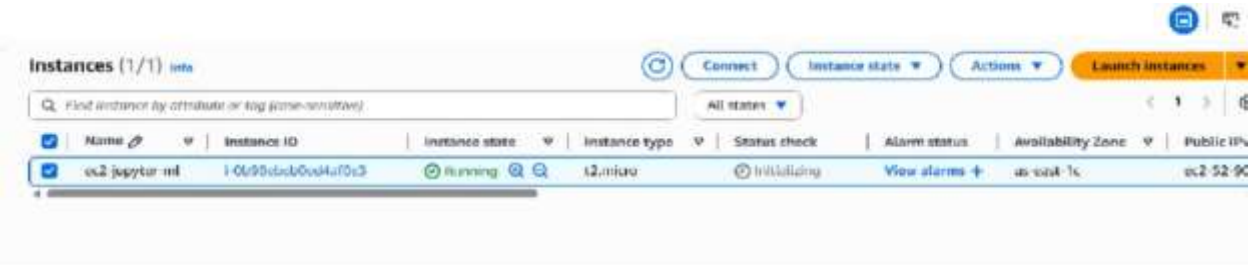
2. Click **Launch instance**

Instance details:

- **Name:** `ec2-jupyter-ml`
- **AMI:** *Amazon Linux 2 (Free tier eligible)*
- **Instance type:** `t2.micro`
- **Key pair:**
  Create new key pair
  - Name: `ec3`
  - Type: RSA
  - Download `.pem` file
- **Network settings:**
  - Allow **SSH (22)**
  - Allow **Custom TCP 8888** (for Jupyter)

3. Click **Launch instance**

EC2 will be launched



STEP 2: Connect to EC2

1. EC2 → Instances
2. Select instance
3. Click **Connect**
4. Choose **EC2 Instance Connect**
5. Click **Connect**

Terminal will open



STEP 3: Install Python & Jupyter Update

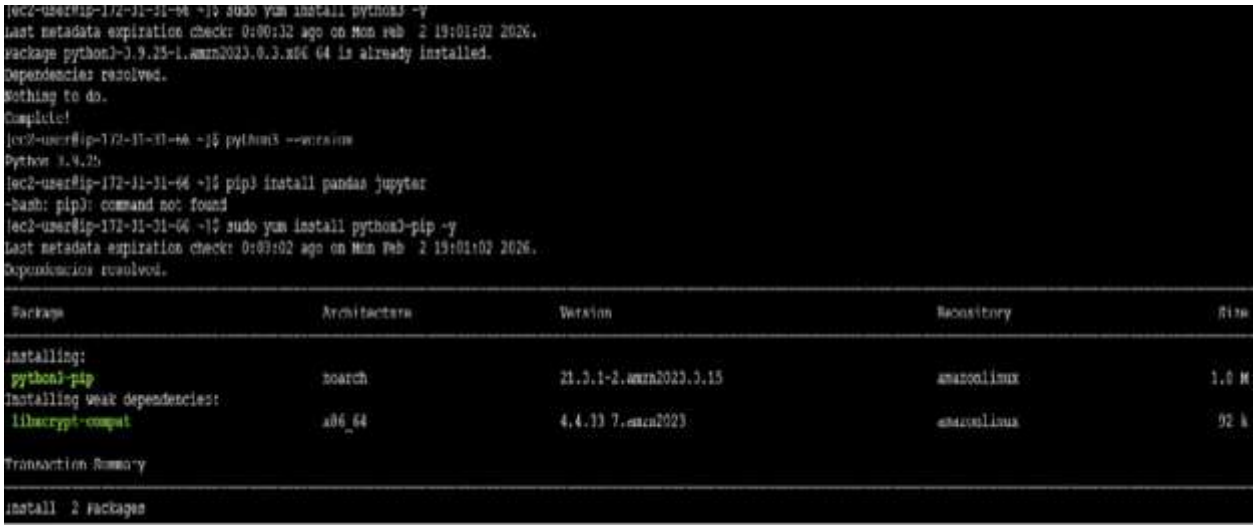
system

```
sudo yum update -y
```

Install Python 3 & pip

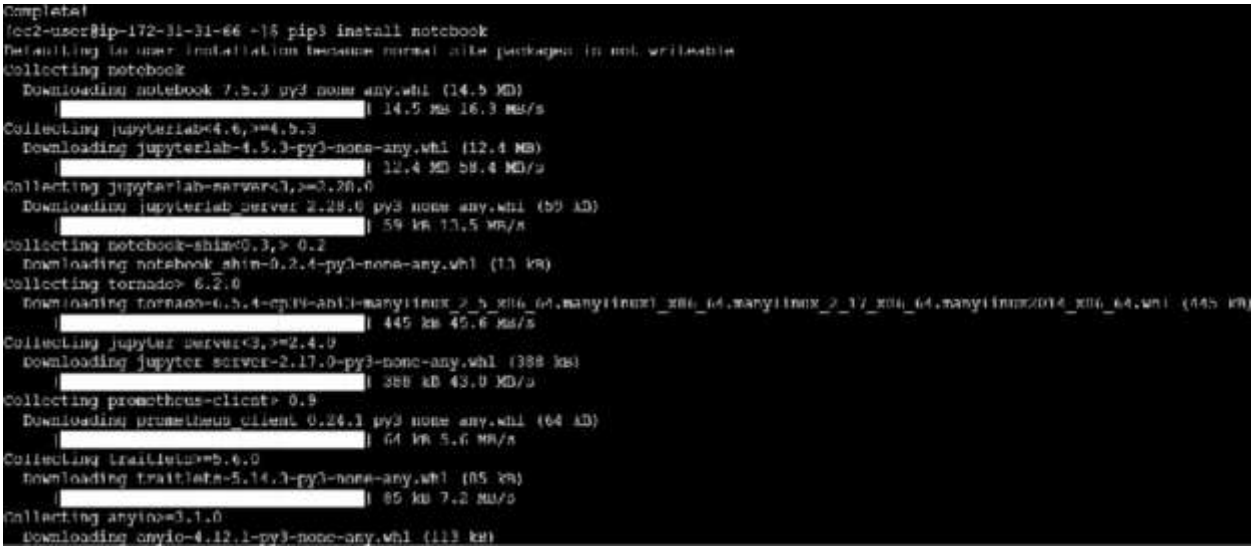
```
sudo yum install python3 -y python3 --version
```

```
sudo yum install python3-pip -y pip3 --version
```



Install Jupyter Notebook

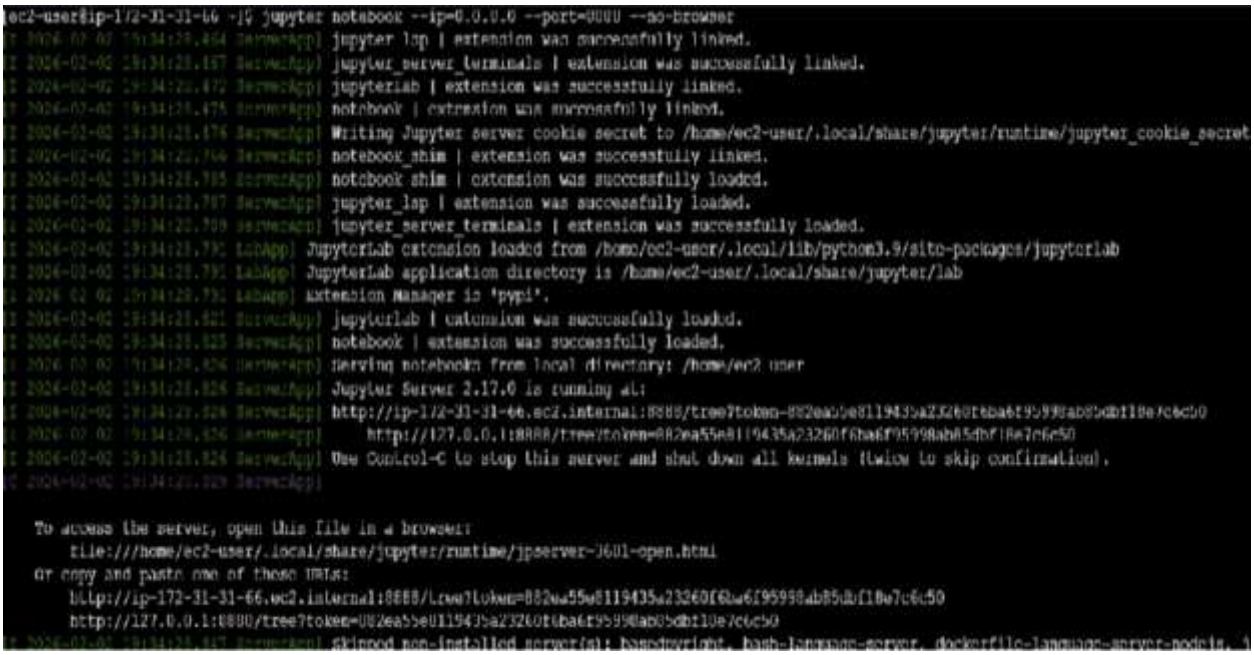
```
pip3 install notebook
```



STEP 4: Start Jupyter Notebook

```
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser Token:
```

```
http://127.0.0.1:8888/?token=abc123...
```



STEP 5: Open Jupyter in Browser

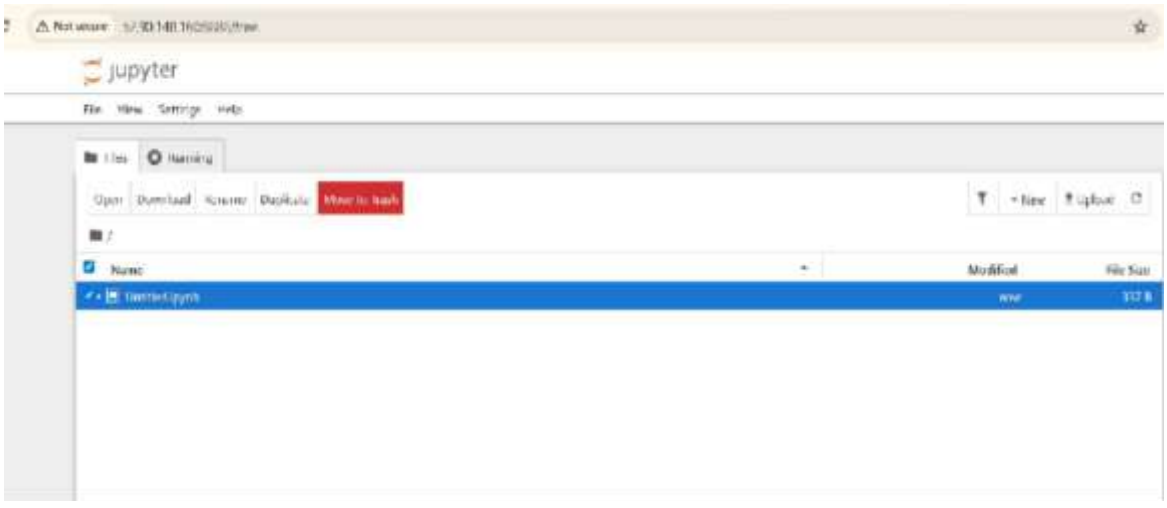
Public IP :

```
54.235.36.14
```

Open the Browser: `http://52.90.148.160:8888/tree`

Paste Token

JUPYTER NOTEBOOK WILL OPEN



Step 6: Load Dataset

```
import pandas as pd import numpy as

np

data = pd.read_csv("enjoysport.csv") data
```

[3]:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes
4	Sunny	Warm	Normal	Weak	Warm	Same	Yes
5	Rainy	Cold	Normal	Weak	Cool	Same	No

Step 7: Implement FIND-S Algorithm

```
# Separate attributes and target

concepts = np.array(data)[:,:-1]

Concepts
```

```
[4]: array([['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'],
            ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'],
            ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change'],
            ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change'],
            ['Sunny', 'Warm', 'Normal', 'Weak', 'Warm', 'Same'],
            ['Rainy', 'Cold', 'Normal', 'Weak', 'Cool', 'Same']], dtype=object)
```

```
target = np.array(data)[:,-1] target
```

```
[6]: array(['Yes', 'Yes', 'No', 'Yes', 'Yes', 'No'], dtype=object)
```

```
# Initialize hypothesis

hypothesis = concepts[0].copy() print("Initial

Hypothesis:", hypothesis)
```

```
Initial Hypothesis: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
```

FIND-S Algorithm

```
for i, example in enumerate(concepts): if target[i]
    == "Yes":
        for j in range(len(hypothesis)): if
            hypothesis[j] == 'Ø':
                hypothesis[j] = example[j] elif
            hypothesis[j] != example[j]:
                hypothesis[j] = '?' print("Final
Hypothesis:", hypothesis)
```

```
Final Hypothesis: ['Sunny' 'Warm' '?' '?' '?' '?']
```

Result

The FIND-S algorithm was successfully implemented using AWS services. The final hypothesis correctly represents the most specific concept learned from the training data stored in Amazon EC2.

EXPERIMENT-2

Aim: To implement the Candidate Elimination algorithm using AWS services and generate the version space by maintaining Specific (S) and General (G) hypothesis boundaries.

Apparatus / Software

- AWS Account (Free Tier)
- **Amazon EC2 (Amazon Linux 2)**
- Python 3.x
- Jupyter Notebook
- CSV Dataset (enjoysport_ce.xlsx)

WS Services Used

- **Amazon EC2** – Execution environment for machine learning code
- **AWS IAM** – Secure access control for EC2 instance

Theory

Candidate Elimination is a **supervised concept learning algorithm** that finds the complete **version space** of hypotheses consistent with the given training data.

Unlike FIND-S, it considers **both positive and negative examples**. The algorithm maintains:

- **Specific Boundary (S):** Most specific hypotheses consistent with the data
- **General Boundary (G):** Most general hypotheses consistent with the data

As more examples are processed, inconsistent hypotheses are eliminated, reducing the version space.

Algorithm

1. Initialize **S** to the most specific hypothesis (Ø values).
2. Initialize **G** to the most general hypothesis (all ?).
3. For each training example:
    - **If positive:**
        - Remove inconsistent hypotheses from G.
        - Generalize S minimally.
    - **If negative:**
        - Remove inconsistent hypotheses from S.
            - Specialize G minimally.
4. Repeat until all examples are processed.
5. Output final S and G boundaries.

Detailed AWS-Based Procedure(Using EC2 Only)

STEP 1: Launch EC2 Instance

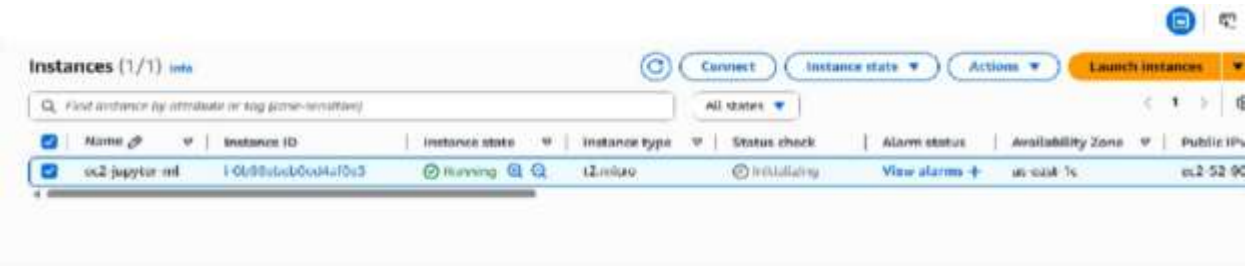
1. AWS Console → **EC2**
2. Click **Launch instance**

Instance details:

- **Name:**`ec2-jupyter-ml`
- **AMI:** *Amazon Linux 2 (Free tier eligible)*
- **Instance type:**`t2.micro`
- **Key pair:**
  Create new key pair
  - Name: `ec3`
  - Type: RSA
  - Download `.pem` file
- **Network settings:**
  - Allow **SSH (22)**
  - Allow **Custom TCP 8888** (for Jupyter)

3. Click **Launch instance**

EC2 Will be launched



STEP 2: Connect to EC2

1. EC2 → Instances
2. Select instance
3. Click **Connect**
4. Choose **EC2 Instance Connect**
5. Click **Connect**

Terminal will open



STEP 3: Install Python & Jupyter

Update system

```
sudo yum update -y
```

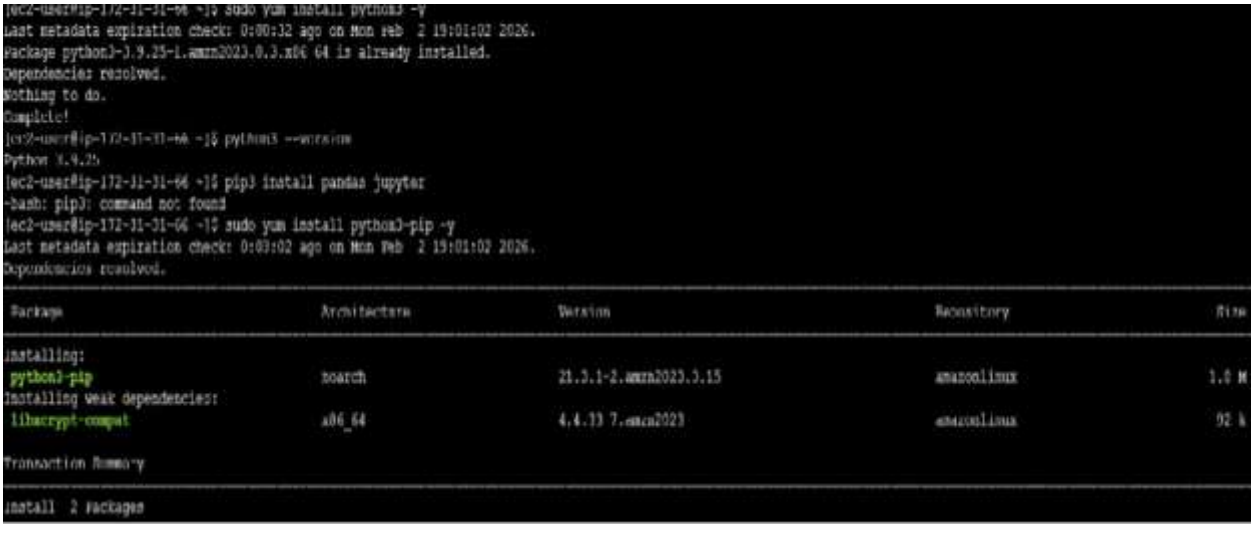
Install Python 3 & pip

```
sudo yum install python3 -y python3
```

```
--version
```

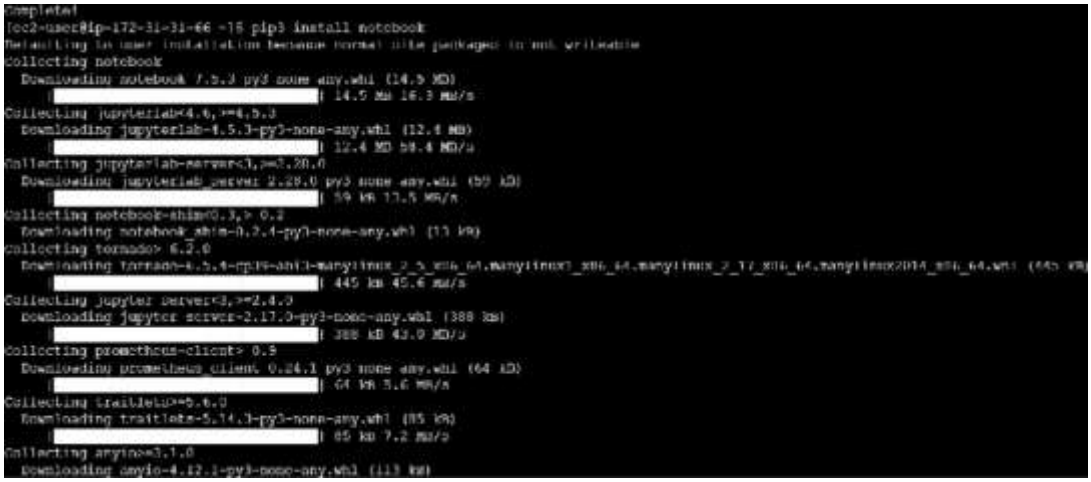
```
sudo yum install python3-pip -y pip3
```

```
--version
```



Install Jupyter Notebook

```
pip3 install notebook
```

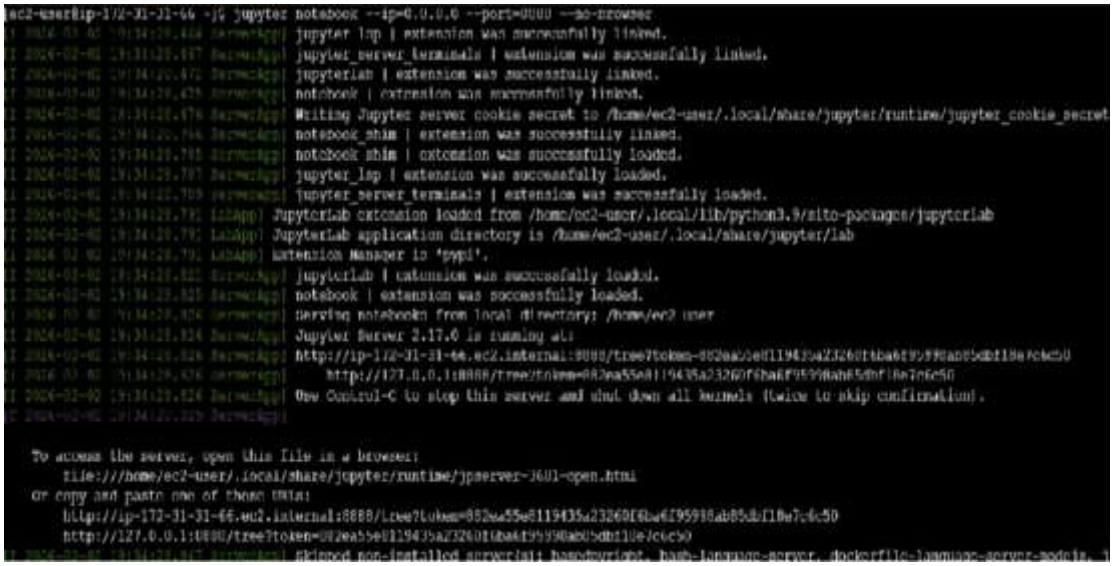


STEP 4: Start Jupyter Notebook

```
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser
```

```
Token:
```

```
http://127.0.0.1:8888/?token=abc123...
```



STEP 5: Open Jupyter in Browser

Public IP :

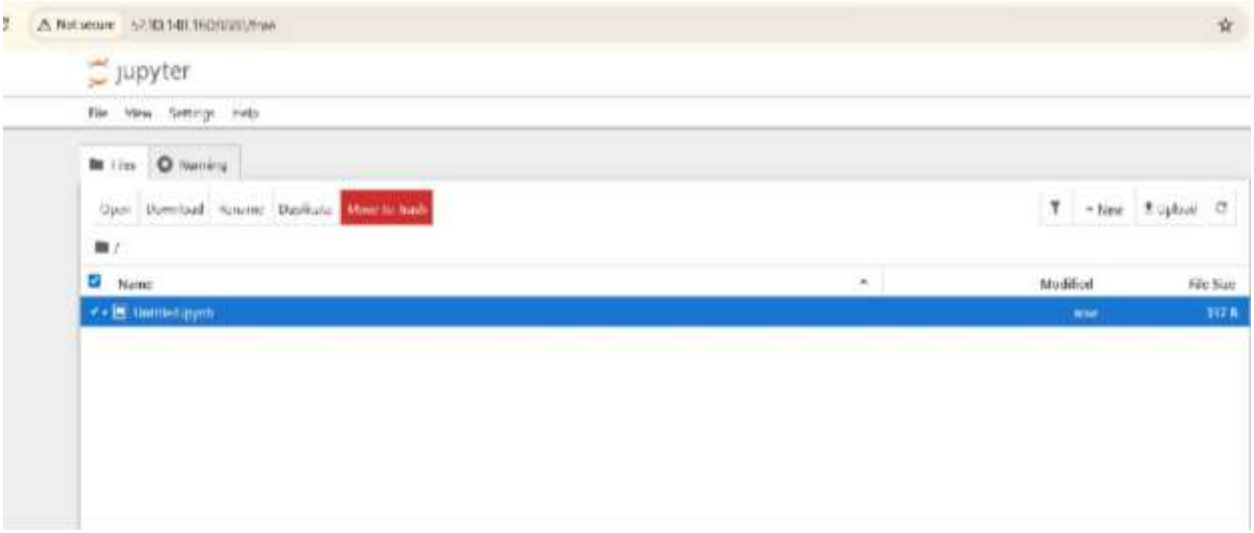
```
54.235.36.14
```

Open the Browser:

```
http://52.90.148.160:8888/tree
```
Paste

Token

JUPYTER NOTEBOOK WILL OPEN



Step 6: Load Dataset

```
import pandas as pd

df = pd.read_excel("enjoysport_ce.xlsx") df
```

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes

Step 7: Initialize Hypothesis Boundaries

```
concepts = df.iloc[:, :-1].values

target = df.iloc[:, -1].values

S = concepts[0].copy()

G = [["?" for _ in range(len(S))]]

print("Initial S:", S) print("Initial

G:", G)
```

```
Initial S: ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
Initial G: [['?', '?', '?', '?', '?', '?']]
```

Step 8: Process Training Examples

```
for i, example in enumerate(concepts):

if target[i] == "Yes":

for j in range(len(S)):

if example[j] != S[j]:

S[j] = "?"

G = [g for g in G if all(g[j] == "?" or g[j] == example[j] for j in
range(len(S)))]

else: new_G =

[]

for g in G:

for j in range(len(S)):

if g[j] == "?" and example[j] != S[j]:

temp = g.copy() temp[j]

= S[j]

new_G.append(temp)

G = new_G print("Final

S:", S)

print("Final G:", G)
```



Result

The Candidate Elimination algorithm was successfully implemented using AWS services. The final Specific and General boundaries accurately represent the version space consistent with the given training data.

EXPERIMENT-3

Aim: To construct and analyze a Decision Tree classifier using the ID3 algorithm by computing entropy and information gain, and to implement the solution using AWS services.

Apparatus / Software Required

- AWS Account (Free Tier)
- Amazon EC2 (Amazon Linux)
- Python 3.x
- Jupyter Notebook
- CSV Dataset (`playtennis.csv`)
- Web Browser / SSH Client

AWS Services Used

- **Amazon EC2** – Execution and visualization of machine learning code
- **AWS IAM** – Secure access control for EC2 instance

Theory

The **ID3 (Iterative Dichotomiser 3)** algorithm is a supervised machine learning algorithm used to generate a **decision tree** from a dataset.

ID3 uses:

- **Entropy** to measure impurity in the dataset
- **Information Gain (IG)** to select the best attribute for splitting

The attribute with the **highest information gain** becomes the decision node.
The process is applied recursively until the dataset is completely classified or no attributes remain.

Entropy Formula

$$Entropy(S) = -p^{+}log\quad_2(p^{+}) - p^{-}log\quad_2(p^{-})$$

Information Gain

$$IG(S,A) = Entropy(S) - \sum_{|S|} |S_v| \times Entropy(S_v)$$

Algorithm

1. Calculate entropy of the target attribute.
2. Compute information gain for each attribute.
3. Select the attribute with maximum information gain.
4. Split the dataset based on the selected attribute.
5. Repeat recursively for each subset until stopping condition is met.

Detailed AWS-Based Procedure(Using EC2 Only)

STEP 1: Launch EC2 Instance

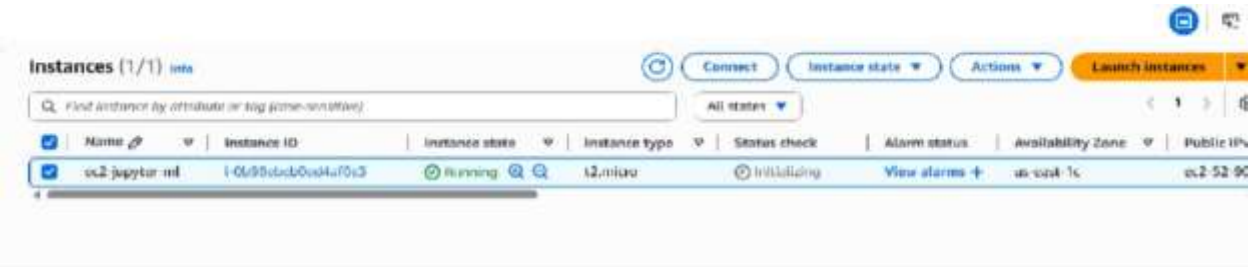
1. AWS Console → **EC2**
2. Click **Launch instance**

Instance details:

- **Name:**`ec2-jupyter-ml`
- **AMI:** *Amazon Linux 2 (Free tier eligible)*
- **Instance type:**`t2.micro`
- **Key pair:**
  Create new key pair
  - Name: `ec3`
  - Type: RSA
  - Download `.pem` file
- **Network settings:**
  - Allow **SSH (22)**
  - Allow **Custom TCP 8888** (for Jupyter)

3. Click **Launch instance**

EC2 Will be launched



STEP 2: Connect to EC2

1. EC2 → Instances
2. Select instance
3. Click **Connect**
4. Choose **EC2 Instance Connect**
5. Click **Connect**

Terminal will open



STEP 3: Install Python & Jupyter Update

system

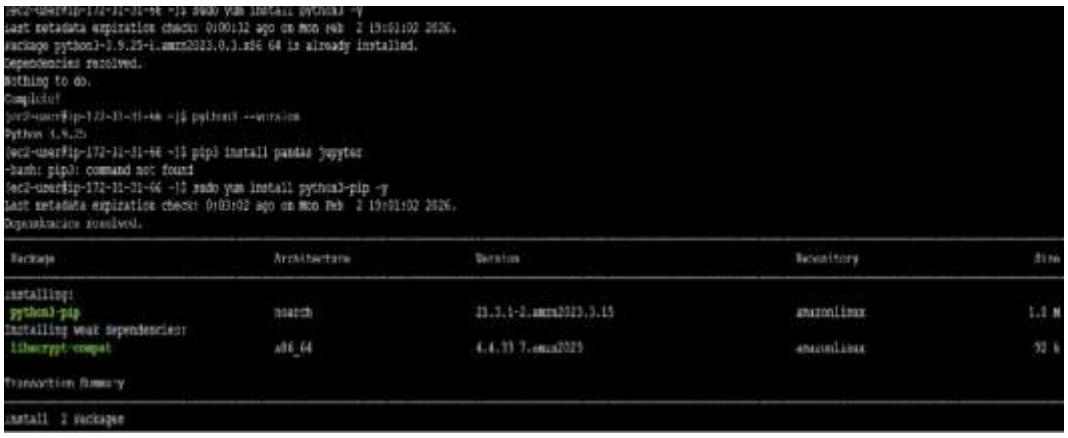
```
sudo yum update -y
```

Install Python 3 & pip

```
sudo yum install python3 -y python3 --
```

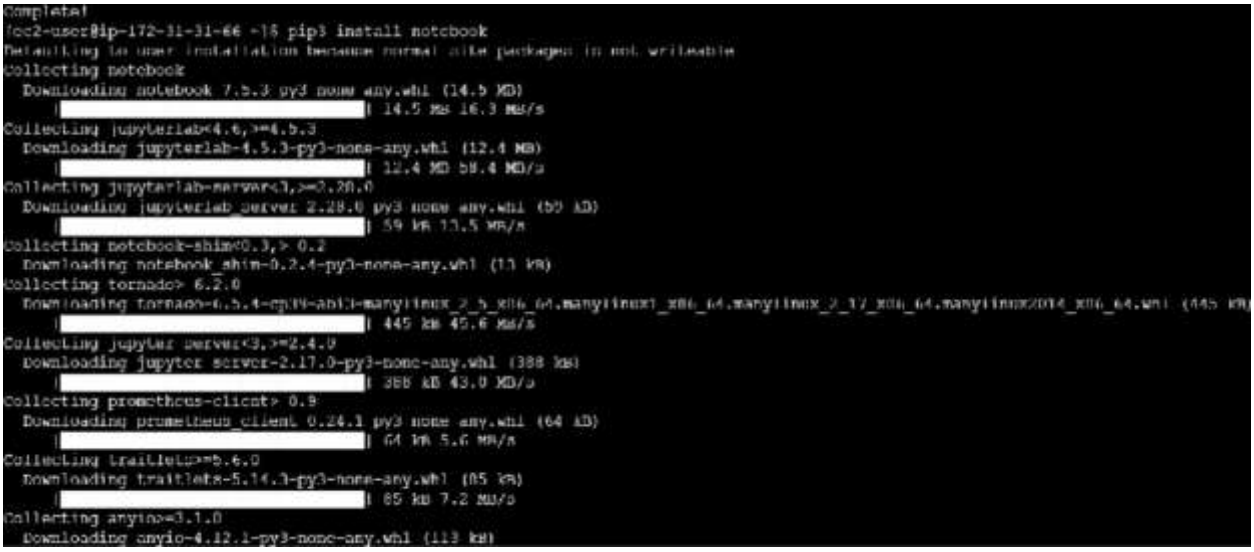
```
version
```

```
sudo yum install python3-pip -y pip3 –version
```



Install Jupyter Notebook

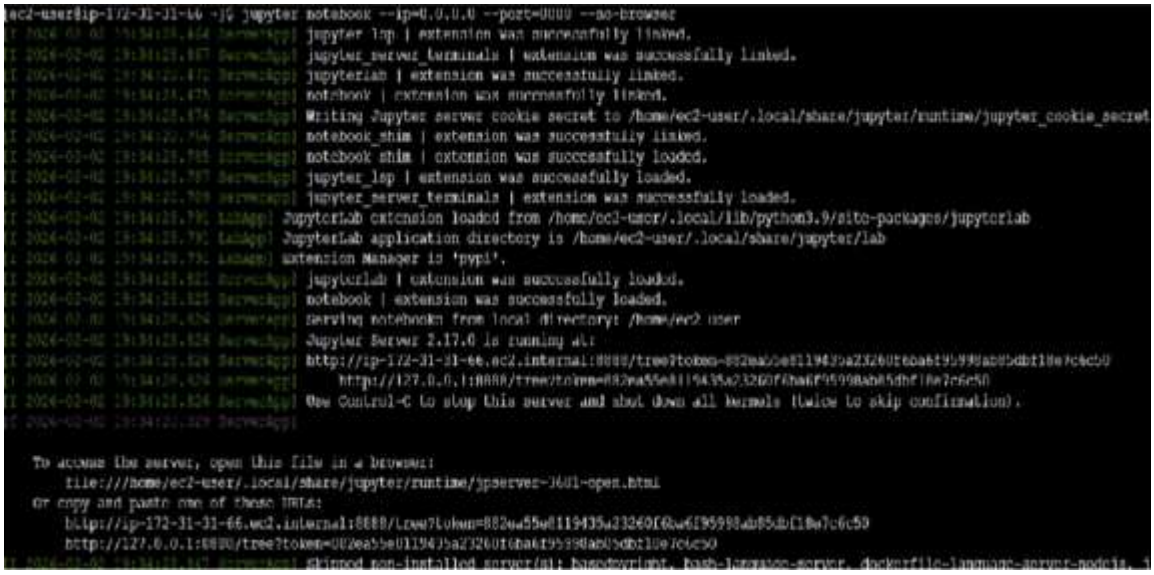
```
pip3 install notebook
```



STEP 4: Start Jupyter Notebook

`jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser Token:`

`http://127.0.0.1:8888/?token=abc123..`



STEP 5: Open Jupyter in Browser

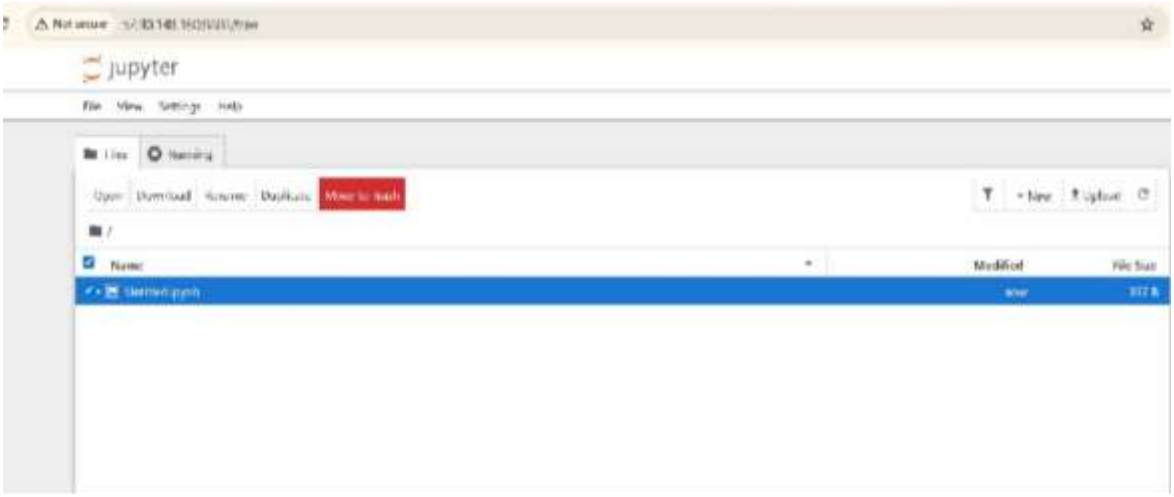
Public IP :

`54.235.36.14`

Open the Browser:

`http://52.90.148.160:8888/tree` Paste Token

JUPYTER NOTEBOOK WILL OPEN



Step 6: Load Dataset

6.1 Load CSV File into DataFrame

```
import pandas as pd

df = pd.read_csv("playtennis.csv") df
```

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No

6.1 Separate Features and Target

```
X = df.iloc[:, :-1]       # Features

y = df.iloc[:, -1]        # Target (PlayTennis)

print("Features:\n", X)

print("\nTarget:\n", y)
```

```
Features:
     Outlook Temperature Humidity    Wind
0      Sunny         Hot     High    Weak
1      Sunny         Hot     High  Strong
2   Overcast         Hot     High    Weak
3       Rain        Mild     High    Weak
4       Rain        Cool   Normal    Weak
5       Rain        Cool   Normal  Strong

Target:
 0     No
1     No
2    Yes
3    Yes
4    Yes
5     No
Name: PlayTennis, dtype: object
```

Step 7: Calculate Entropy of Target Attribute

7.1 Count Positive & Negative

```
df['PlayTennis'].value_counts()
```

```
[6]: PlayTennis
     No     3
     Yes    3
     Name: count, dtype: int64
```

7.2 Entropy Formula

$$Entropy(S) = -p^{+}log_{2}(p^{+}) - p^{-}log_{2}(p^{-})$$

$$Entropy(S) = -0.5log_{2}(0.5) - 0.5log_{2}(0.5)$$

$$\text{Entropy(S)}=1.0$$

7.3 Entropy Using Python

```
import numpy as np def

entropy(col):

values, counts = np.unique(col, return_counts=True) probs = counts

/ counts.sum()

return -np.sum(probs * np.log2(probs)) entropy_S =

entropy(df['PlayTennis']) entropy_S
```

```
[8]: np.float64(1.0)
```

Step 8: Compute Information Gain

8.1 Information Gain Function

```
def information_gain(data, feature, target):

total_entropy = entropy(data[target])

values, counts = np.unique(data[feature], return_counts=True)

weighted_entropy = 0

for v, c in zip(values, counts):

subset = data[data[feature] == v]

weighted_entropy += (c / len(data)) * entropy(subset[target])
```

```
return total_entropy - weighted_entropy
```

8.2 Compute IG for Each Attribute

```
for feature in df.columns[:-1]:

ig = information_gain(df, feature, 'PlayTennis')

print(f"{feature} : {ig:.4f}")
```

```
Outlook : 0.5409
Temperature : 0.2075
Humidity : 0.0000
Wind : 0.4591
```

Step 9: Select Best Attribute and Split

Since **Outlook has maximum IG**, **Root Node =**

Outlook

Tree structure:

```
Outlook

├── Sunny → No

├── Overcast → Yes

└── Rain → Wind

                ├── Strong → No

                └── Weak → Yes
```

Step 10: Build Decision Tree Recursively

ID3 Recursive Function

```
def id3(data, target, features):

if len(np.unique(data[target])) == 1:

return data[target].iloc[0] if

len(features) == 0:

return data[target].mode()[0]

gains = {f: information_gain(data, f, target) for f in features}

best_feature = max(gains, key=gains.get)

tree = {best_feature: {}}

remaining_features = [f for f in features if f != best_feature]
```

```
for value in np.unique(data[best_feature]):

subset = data[data[best_feature] == value]

tree[best_feature][value] = id3(subset, target, remaining_features)

return tree
```

Generate Tree

```
features = list(df.columns[:-1])

decision_tree = id3(df, 'PlayTennis', features)

decision_tree
```

```
[13]: {'Outlook': {'Overcast': 'Yes',
                   'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}},
                   'Sunny': 'No'}}
```

Step 11: Classify New Sample Test

Sample:

⟨Sunny, Cool, High, Strong⟩ Decision:

- Outlook = Sunny
- Sunny branch → **No**

```
print("Predicted Output: No")
```

```
Predicted Output: No
```

Result

The Decision Tree was successfully constructed using the ID3 algorithm in an AWS EC2 environment. The model correctly classified unseen samples based on entropy and information gain.

EXPERIMENT 4

K-NEAREST NEIGHBORS (KNN) ALGORITHM

Aim:To run a K-Nearest Neighbors (KNN) machine-learning example on AWS EC2 using Jupyter Notebook

Learning Outcomes
After completing this experiment, students will be able to:

1. Launching EC2
2. Installing Jupyter & libraries
3. Running a full KNN example in a notebook
4. Testing from your browser

Theory: KNN

K-Nearest Neighbors (KNN) is one of the simplest and most intuitive supervised machine learning algorithms used for:

- Classification
- Regression

It is called a lazy learning algorithm because it does not build a model during training. Instead, it stores the dataset and makes predictions only when a new query arrives.

1. Basic Idea of KNN

"A data point is classified based on how its nearest neighbors are classified."

If most of the nearest data points belong to class **A**, the new point will also be labeled as **A**.
Real-life intuition

If you want to guess:

- the favorite movie of a person → look at preferences of similar friends
- credit card fraud → compare with most similar past transactions

2. How KNN Works – Core Concept

For a new input sample:

1. Measure distance between new point and all training samples
2. Select **K nearest points**
3. Take majority vote (for classification)
4. Take average (for regression)

3. Mathematical Explanation of KNN

KNN is a **distance-based, non-parametric** learning algorithm.
For a query point

$$x = (x_1, x_2, \ldots, x_n)$$

and a training sample

$$x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)})$$

Distance Computation

(a) Euclidean Distance

$$d(x, x^{(i)}) = \sqrt{\sum_{j=1}^{n} (x_j - x_j^{(i)})^2}$$

(b) Manhattan Distance

$$d(x, x^{(i)}) = \sum_{j=1}^{n} |x_j - x_j^{(i)}|$$

(c) Minkowski Distance

$$d(x, x^{(i)}) = \left(\sum_{j=1}^{n} |x_j - x_j^{(i)}|^p \right)^{1/p}$$

- p = 1 → Manhattan
- p = 2 → Euclidean

Classification Rule

Let

$$N_k(x)$$

be the set of k nearest neighbors of x.

Predicted class:

$$\hat{y} = \text{mode}\{y^{(i)} : x^{(i)} \in N_k(x)\}$$

Regression Rule

$$\hat{y} = \frac{1}{k} \sum_{x^{(i)} \in N_k(x)} y^{(i)}$$

Weighted KNN

Closer neighbors get more importance:

$$w_i = \frac{1}{d(x, x^{(i)})}$$

$$\hat{y} = \text{argmax} \sum w_i \cdot I(y = y^{(i)})$$

4. Steps of KNN Algorithm

Step 1 – Choose the value of K

- K = number of neighbors
- Usually odd (3,5,7…) to avoid ties

Step 2 – Calculate Distance

Most commonly used distance metrics:

Euclidean Distance

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Manhattan Distance

$$d = \sum_{i=1}^{n} |x_i - y_i|$$

Step 3 – Find K Nearest Neighbors

- Sort all distances in ascending order
- Select top K closest samples

Step 4 – Voting / Averaging

- Classification → majority voting
- Regression → mean of neighbors

Step 5 – Assign Class Label

Return predicted class for the new instance.

Comparison of KNN Types

5. Advantages and Limitations

Advantages

1. Simple & Intuitive – easy to understand
2. No training phase – lazy learner
3. Works for both classification & regression
4. Non-parametric – no assumption about data distribution
5. Adapts to complex decision boundaries

Limitations

1. High computation cost at prediction time
2. Requires storing full dataset → high memory

3. Sensitive to:
    - irrelevant features
    - noisy data
4. Needs feature scaling
5. Suffers from curse of dimensionality
6. Choice of K is critical

6. Types of KNN

(1) Based on Task

Classification KNN

- Output → class label
- Uses majority voting

Regression KNN

- Output → numeric value
- Uses mean/median

(2) Based on Weight

Uniform KNN

- All neighbors equal weight

Distance Weighted KNN

- Closer → more weight

(3) Based on Distance Metric

- Euclidean KNN
- Manhattan KNN
- Minkowski KNN
- Hamming KNN (categorical)

(4) Based on Search Strategy

- Brute Force KNN
- KD-Tree KNN
- Ball Tree KNN
- Approximate KNN

7. Comparison of KNN Types

Type	Idea	Pros	Cons
Classification KNN	Majority vote	Simple	Tie problem
Regression KNN	Average	Smooth output	Sensitive to outliers
Weighted KNN	Weight by distance	More accurate	Extra computation
Brute Force	Compare all points	Exact	Very slow
KD Tree	Space partition	Fast low-dim	Poor in high-dim
Ball Tree	Hypersphere	Better than KD	Complex
Hamming KNN	Categorical	Works on text	Limited metrics

When to Use?

- Small dataset → Brute force
- Low dimension → KD-Tree
- High dimension → Weighted / Approximate
- Categorical → Hamming distance
- Noisy data → larger K

Implementation Steps

1. Launch an EC2 Instance

Step 1: Create EC2

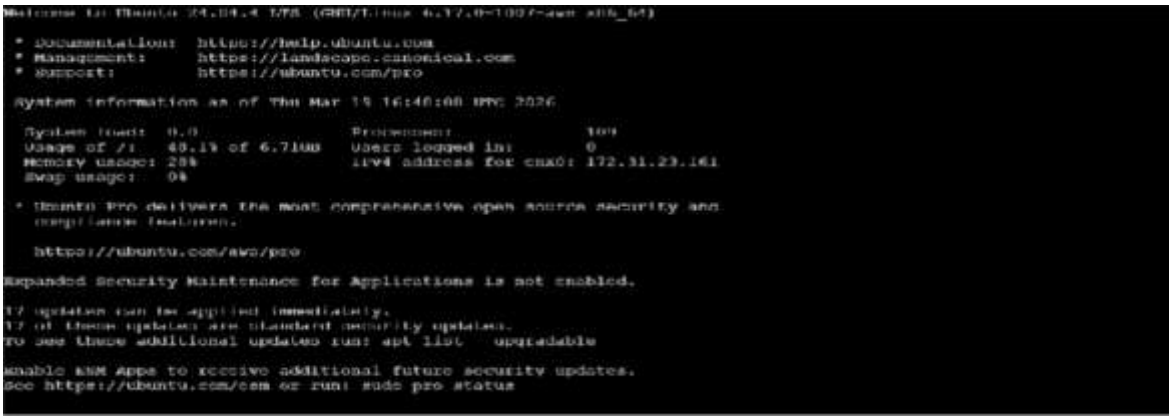
- Login to AWS → **EC2 Dashboard** → *Launch Instance*



- Choose:
  - **AMI:** Ubuntu Server 22.04 (recommended)
  - **Instance type:** t2.micro (free tier)
  - Key pair: create or use existing
  - Security Group: allow
    - SSH – 22
    - Custom TCP – **8888** (for Jupyter)

Step 2: Connect via SSH

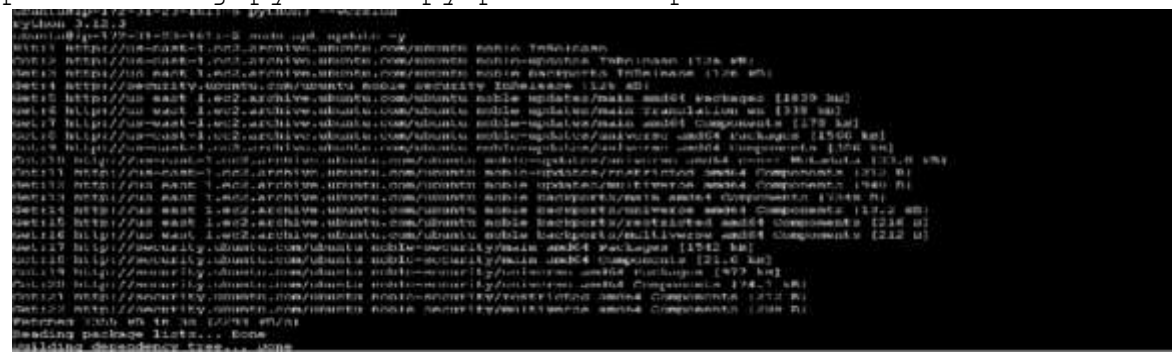
```
ssh -i yourkey.pem ubuntu@your-ec2-public-ip
```



2. Install Python & Jupyter

Run these commands on EC2 terminal:

```
sudo apt update
sudo apt install python3-pip -y
pip3 install jupyter numpy pandas matplotlib scikit-learn
```



Generate Jupyter config:

```
jupyter notebook --generate-config
```

Create password:

```
python3 -c "from notebook.auth import passwd;
print(passwd())"
```

Copy generated hash.

Edit config:

```
nano ~/.jupyter/jupyter_notebook_config.py
```

Add:

```
c = get_config()
c.NotebookApp.ip = '0.0.0.0'
c.NotebookApp.port = 8888
c.NotebookApp.open_browser = False
c.NotebookApp.password = 'paste_generated_hash_here'
```

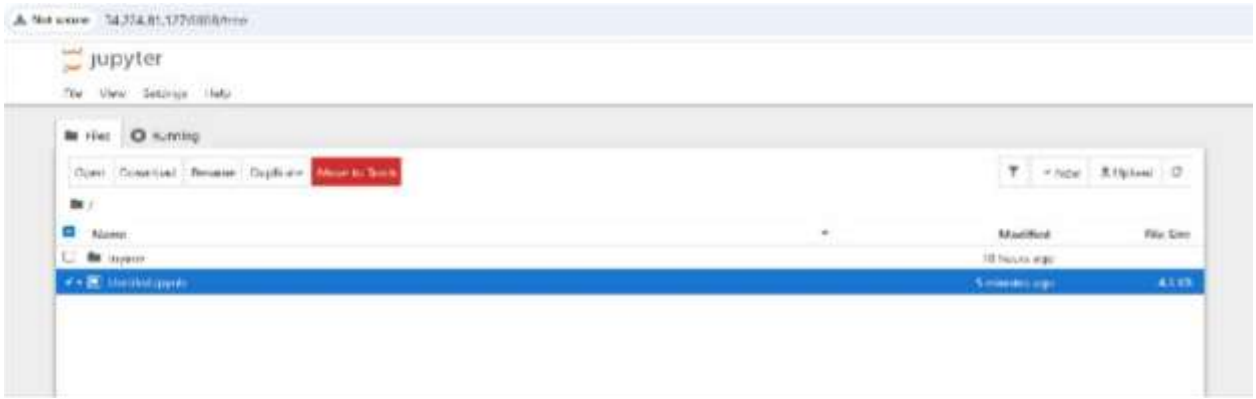
Start notebook:

```
jupyter notebook
```



Open in browser:

http://EC2_PUBLIC_IP:888



3. Create KNN Example in Jupyter
Create new notebook

Import Libraries

```
import numpy as np import
pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier from
sklearn.metrics import accuracy_score,
classification_report, confusion_matrix
```

Load Dataset

```
iris = load_iris()
X = iris.data y =
iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df['target'] = y

df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split( X,
    y, test_size=0.2, random_state=42
)

print("Training samples:", len(X_train)) print("Testing
samples:", len(X_test))
```

```
Training samples: 120
Testing samples: 30
```

Train KNN Model

```
knn = KNeighborsClassifier(n_neighbors=5) knn.fit(X_train, y_train)
```



Prediction

```
y_pred = knn.predict(X_test)
print("Predictions:", y_pred)
print("\nCorrect and Wrong Predictions:")
correct = 0
wrong = 0
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    if y_test[i] == y_pred[i]:
        print(f"Test Sample {i+1}: Actual = {actual}, Predicted =
{predicted} ->CORRECT")
        correct += 1
    else:
        print(f"Test Sample {i+1}: Actual = {actual}, Predicted =
{predicted} -> WRONG")
        wrong += 1
print("\nTotal Correct Predictions:", correct)
print("Total Wrong Predictions:", wrong)
```

```
------------------------------------------------------------
Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

------------------------------------------------------------
CORRECT AND WRONG PREDICTIONS
------------------------------------------------------------
Test Sample 1: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 2: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 3: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 4: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 5: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 6: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 7: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 8: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 9: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 10: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 11: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 12: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 13: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 14: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 15: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 16: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 17: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 18: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 19: Actual = versicolor, Predicted = versicolor --> CORRECT
Test Sample 20: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 21: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 22: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 23: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 24: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 25: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 26: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 27: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 28: Actual = virginica, Predicted = virginica --> CORRECT
Test Sample 29: Actual = setosa, Predicted = setosa --> CORRECT
Test Sample 30: Actual = setosa, Predicted = setosa --> CORRECT
```

Evaluate Model

```
acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc) print("\nClassification
Report:\n") print(classification_report(y_test,
y_pred)) print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 1.0

Classification Report:

              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


Confusion Matrix:

[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

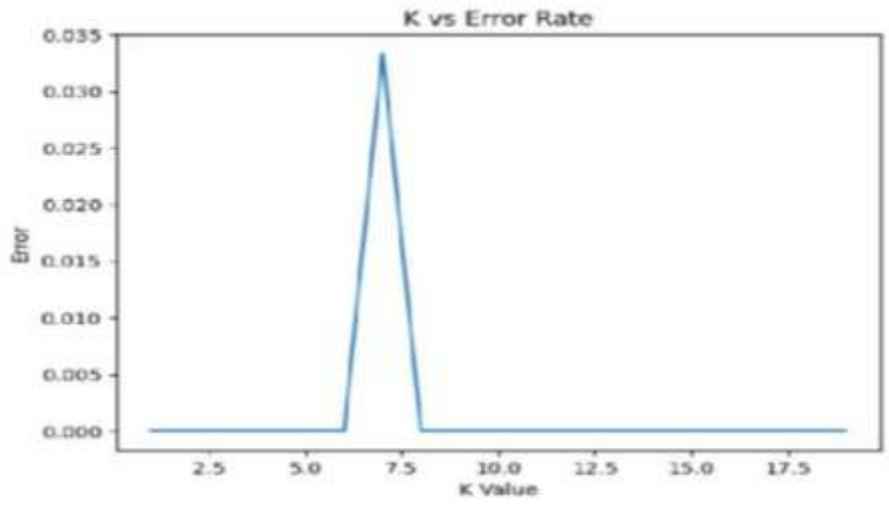
Try Custom Input

```
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = knn.predict(sample)
print("Predicted Class:", iris.target_names[prediction])
```

```
Predicted Class: ['setosa']
```

Experiment with Different K

```
error = []
for k in range(1, 20):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred_k = model.predict(X_test) error.append(np.mean(pred_k
    != y_test))
plt.plot(range(1,20), error) plt.xlabel("K Value")
plt.ylabel("Error") plt.title("K vs Error Rate")
plt.show()
```



EXPERIMENT 5 - NAÏVE BAYES CLASSIFIER

Aim: To implement a Naïve Bayes Classifier on tabular data using EC2 and Jupyter and evaluate the model accuracy.

Learning Outcomes
After completing this experiment, students will be able to:

- Understand the working principle of Naïve Bayes classifier
- Use AWS SageMaker for machine learning model development
- Train and test a classification model on tabular data
- Evaluate model performance using accuracy metrics

AWS Services Used

AWS Service	Purpose
Amazon S3	Store dataset
Amazon SageMaker	Model training and evaluation
AWS IAM	Access control
Amazon CloudWatch	Monitor training logs

Naïve Bayes Classifier

What is Classification?
Classification is a supervised machine learning task in which the algorithm learns from labeled training data and predicts the class label for unseen data. Examples include spam vs. non-spam emails, disease vs. healthy patients, etc.

Intuition Behind Naïve Bayes
Naïve Bayes works on the idea that probability can be used to make decisions. It answers the question:
Given a set of feature values, which class is most likely?
The algorithm is called "naïve" because it assumes that all input features are conditionally independent, which simplifies computation and makes the algorithm very fast.

Mathematical Explanation
Bayes' Theorem is given by:
$P(C \mid X) = (P(X \mid C) \times P(C)) / P(X)$
Where:

- **C** → Class label (e.g., Iris-setosa)
- **X** → Input features (sepal length, width, etc.)
- **P(C)** → Prior probability of class
- **P(X | C)** → Likelihood of features given class
- **P(C | X)** → Posterior probability

The class with the highest posterior probability is selected as the prediction.

Why Gaussian Naïve Bayes?
Gaussian Naïve Bayes is used when features are continuous numeric values. It assumes that feature values follow a normal (Gaussian) distribution.
Gaussian Probability Density Function:
$P(x \mid C) = (1 / \sqrt{(2\pi\sigma^2)}) \times \exp(-(x - \mu)^2 / (2\sigma^2))$
Where:

- $\mu \rightarrow$ Mean of feature
- $\sigma^2 \rightarrow$ Variance of feature

Advantages and Limitations

Advantages:

- Simple and fast
- Works well with small datasets
- Requires less training data

Limitations:

- Independence assumption is unrealistic
- Lower accuracy for complex datasets

Naïve Bayes is a probabilistic supervised learning algorithm based on Bayes' Theorem with the assumption of feature independence.

Bayes' Theorem:

$$P(C|X) = (P(X|C) \times P(C)) / P(X)$$

Where:

- $P(C|X)$ = Posterior probability
- $P(X|C)$ = Likelihood
- $P(C)$ = Prior probability
- $P(X)$ = Evidence

Types of Naïve Bayes Classifiers

Naïve Bayes classifiers are categorized based on the nature of input data and the assumptions made about feature distributions. Each type is suitable for specific applications.

1. <u>Gaussian Naïve Bayes</u>

Definition:

Gaussian Naïve Bayes is used when the input features are continuous numerical values and are assumed to follow a normal (Gaussian) distribution.

Assumption:

Each feature within a class follows a bell-shaped curve characterized by mean (μ) and variance (σ^2).

Mathematical Model:

$P(x \mid C) = (1 / \sqrt{(2\pi\sigma^2)}) \times \exp(-(x - \mu)^2 / (2\sigma^2))$

Key Characteristics:

- Works well with real-valued attributes
- No need for data discretization
- Fast training and prediction

Applications:

- Medical diagnosis (e.g., blood pressure, sugar level)
- Iris flower classification
- Sensor data analysis

Example:

In the Iris dataset, features like sepal length and petal width are continuous, hence Gaussian Naïve Bayes is appropriate.

Multinomial Naïve Bayes

Definition:
Multinomial Naïve Bayes is mainly used for discrete count-based data, especially in text classification problems.
Assumption:
Features represent frequency or count of events (e.g., word occurrences).
Mathematical Idea:
It models the probability of feature counts given a class.
Key Characteristics:

- Works on non-negative integer values
- Sensitive to feature frequency
- Often combined with TF-IDF

Applications:

- Spam email detection
- Sentiment analysis
- Document categorization

Example:
Number of times words like "offer", "free", "win" appear in an email.

Bernoulli Naïve Bayes
Definition:
Bernoulli Naïve Bayes is used when features are binary (0/1 or True/False).
Assumption:
Each feature indicates presence or absence of a characteristic.
Key Characteristics:

- Binary-valued features
- Penalizes absence of features
- Suitable for small datasets

Applications:

- Binary text classification
- Click prediction systems
- Yes/No based datasets

Example:
Whether a word appears in an email (1) or not (0).

Complement Naïve Bayes
Definition:
Complement Naïve Bayes is an improvement over Multinomial Naïve Bayes, designed to handle imbalanced datasets.
Key Characteristics:

- Uses statistics from all classes except the current class
- Reduces bias towards dominant classes

Applications:

- Large-scale document classification
- Highly skewed text datasets

Comparison of Naïve Bayes Types

Type	Data Type	Typical Use Case
Gaussian NB	Continuous	Medical, sensor, numeric data
Multinomial NB	Count-based	Text classification
Bernoulli NB	Binary	Presence/absence data
Complement NB	Imbalanced text	Large document corpora

In this experiment, Gaussian Naïve Bayes is used because the dataset consists of continuous numerical features.

5. Dataset Description (Example)

Dataset Used: Iris Dataset (Tabular Data)

Feature	Description
SepalLength	Length of sepal
SepalWidth	Width of sepal
PetalLength	Length of petal
PetalWidth	Width of petal
Species	Class label

System Requirements

Hardware

- PC/Laptop with Internet

Software

- AWS Account
- Python 3.x
- AWS SageMaker Notebook Instance

7. Step-wise Procedure (AWS Implementation)

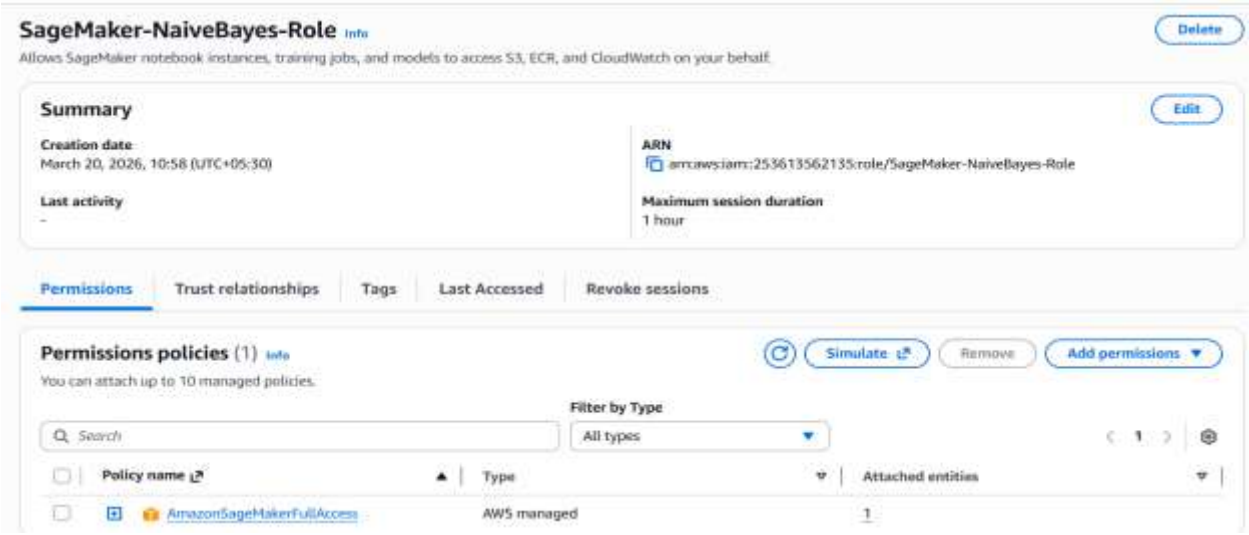
This experiment is implemented using AWS SageMaker, which provides a managed Jupyter Notebook environment and scalable ML infrastructure.

Step 1: AWS Account Setup and IAM Role Creation

Before accessing AWS ML services, proper permissions are required.

Steps:

1. Login to AWS Management Console
2. Navigate to IAM → Roles → Create Role



3. Choose AWS Service → SageMaker
4. Attach the following policies:
    - AmazonSageMakerFullAccess
    - AmazonS3FullAccess
5. Name the role: SageMaker-NaiveBayes-Role

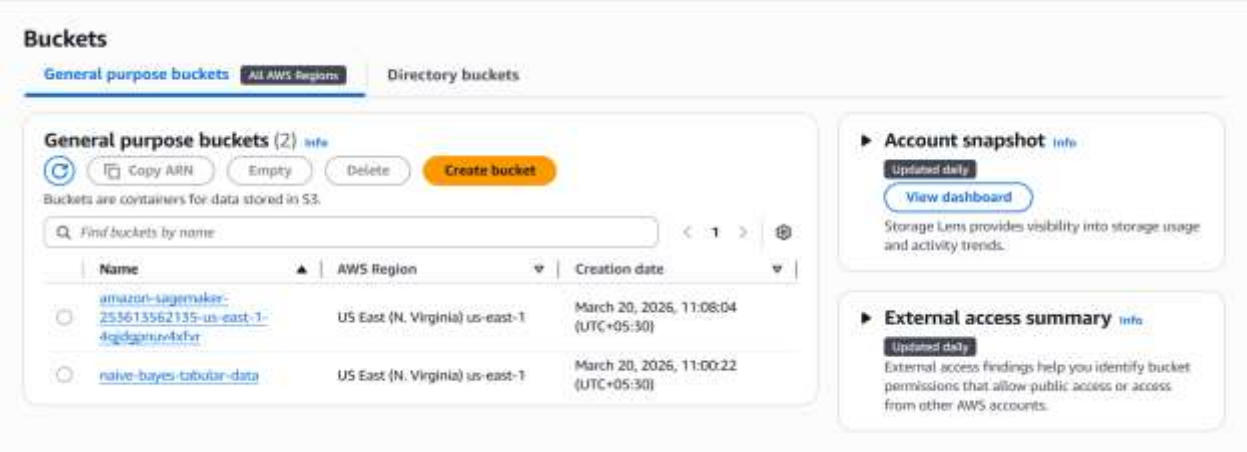
Purpose: IAM role allows SageMaker to securely access S3 datasets and training resources.

Step 2: Dataset Upload to Amazon S3

Amazon S3 is used as a central storage location for datasets.

Steps:

1. Open **Amazon S3** service
2. Create a bucket (e.g., naive-bayes-tabular-data)
3. Upload the iris.csv file



Why S3?

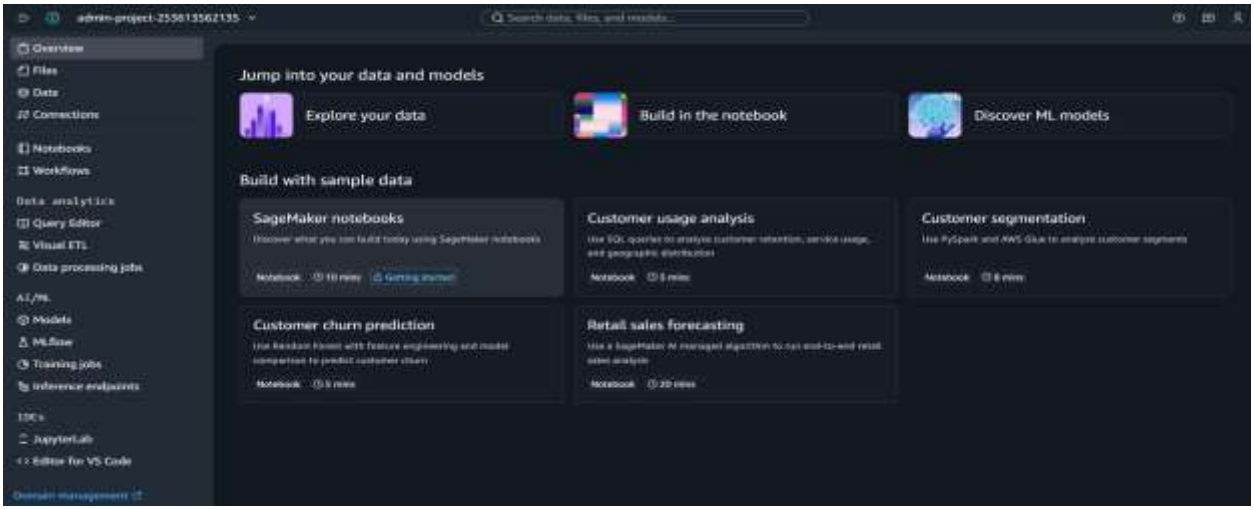
- Durable storage
- Easily accessible by SageMaker
- Scalable for large datasets

Step 3: Launch SageMaker Notebook Instance

Steps:

1. Go to Amazon SageMaker → Notebook instances
2. Click Create notebook instance
3. Instance type: ml.t2.medium
4. Attach IAM role created earlier
5. Click Create → Open JupyterLab

Purpose: Provides a managed Python environment for ML development.



Step 4: Load and Understand the Dataset

```
import pandas as pd
data = pd.read_csv('iris.csv')
print(data.head())
print(data.info())
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

Step 5: Data Preprocessing and Feature-Label Separation

```
X = data.iloc[:, :-1] # Features
y = data.iloc[:, -1] # Target label
```

Step 6: Train-Test Split

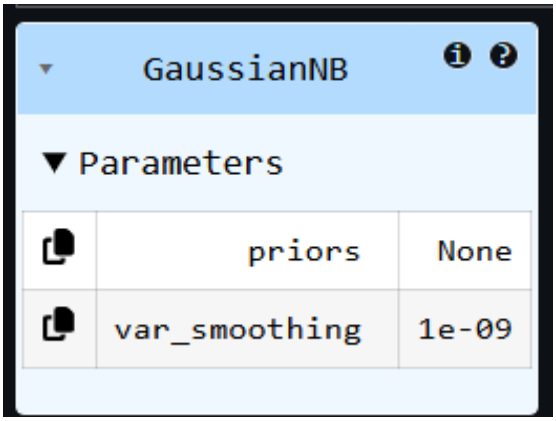
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
```

Explanation:

- 80% data used for training
- 20% data used for testing
- Ensures unbiased model evaluation

Step 7: Model Training Using Gaussian Naïve Bayes

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
```



Explanation:

- Model learns class-wise feature distributions
- Computes mean and variance for each feature

Step 8: Prediction on Test Data

```
y_pred = model.predict(X_test)
print(y_pred)
```

```
['versicolor' 'setosa' 'virginica' 'versicolor' 'versicolor' 'setosa'
 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'setosa'
 'setosa' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'versicolor' 'virginica' 'setosa' 'virginica' 'setosa' 'virginica'
 'virginica' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa']
```

Explanation:

- Model predicts class labels for unseen data

Step 9: Model Evaluation (Accuracy Calculation)

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Explanation: Accuracy measures the ratio of correctly predicted samples to total samples.
Accuracy = (Correct Predictions / Total Predictions)

```
Accuracy: 1.0
```

EXPERIMENT 6 - NAÏVE BAYES CLASSIFIER

Aim: Write a program to classify a set of documents use the naïve Bayesian Classifier model. Calculate the accuracy, precision, and recall for your dataset.

Learning Outcomes

After completing this experiment, students will be able to:

- Understand the working principle of Naïve Bayes classifier.
- Apply the Naïve Bayes classifier to document classification.
- Load and process a document dataset stored in a CSV file.
- Train and test a classification model using AWS SageMaker and JupyterLab.
- Classify unseen documents into appropriate categories.
- Evaluate the classifier using accuracy, precision, and recall.

AWS Services Used

AWS Service	Purpose
Amazon S3	Store dataset
Amazon SageMaker	Model training and evaluation
AWS IAM	Access control
Amazon CloudWatch	Monitor training logs

Naïve Bayes Classifier

What is Classification?

Classification is a supervised machine learning task in which the algorithm learns from labeled training data and predicts the class label for unseen data. Examples include spam vs. non-spam emails, disease vs. healthy patients, etc.

Intuition Behind Naïve Bayes

Naïve Bayes works on the idea that probability can be used to make decisions. It answers the question:
Given a set of feature values, which class is most likely?
The algorithm is called "naïve" because it assumes that all input features are conditionally independent, which simplifies computation and makes the algorithm very fast.

Mathematical Explanation

Bayes' Theorem is given by:
$P(C \mid X) = (P(X \mid C) \times P(C)) / P(X)$
Where:

- **C** → Class label (e.g., Iris-setosa)
- **X** → Input features (sepal length, width, etc.)
- **P(C)** → Prior probability of class
- **P(X | C)** → Likelihood of features given class
- **P(C | X)** → Posterior probability

The class with the highest posterior probability is selected as the prediction.

Why Multinomial Naïve Bayes?

Multinomial Naïve Bayes is commonly used for text and document classification because it works with discrete features such as word counts or word frequencies.
In this experiment, each document is represented using the words contained in it.

For a word w and class C, the probability can be estimated using:
$P(w \mid C) = (\text{Count}(w,C) + 1) / (\text{Total words in C} + \text{Vocabulary size})$
The +1 represents Laplace smoothing.
Laplace smoothing prevents a word that has not appeared in a particular class from producing a probability of zero.

Advantages and Limitations

Advantages:

- Simple and fast
- Simple and fast.
- Easy to implement.
- Works well with small datasets.
- Suitable for text and document classification.
- Requires relatively little training data.
- Computationally efficient.

Limitations:

- Assumes features are conditionally independent.
- The independence assumption may not always be realistic.
- Performance can decrease when the relationship between words is complex.
- Zero-frequency problems can occur without smoothing.

Types of Naïve Bayes Classifiers

Naïve Bayes classifiers are categorized based on the nature of input data and the assumptions made about feature distributions. Each type is suitable for specific applications.

1. <u>Gaussian Naïve Bayes</u>

Definition:
Gaussian Naïve Bayes is used when the input features are continuous numerical values and are assumed to follow a normal (Gaussian) distribution.

Assumption:
Each feature within a class follows a bell-shaped curve characterized by mean (μ) and variance (σ^2).

Mathematical Model:
$P(x \mid C) = (1 / \sqrt{(2\pi\sigma^2)}) \times \exp(-(x - \mu)^2 / (2\sigma^2))$

Key Characteristics:

- Works well with real-valued attributes
- No need for data discretization
- Fast training and prediction

Applications:

- Medical diagnosis (e.g., blood pressure, sugar level)
- Iris flower classification
- Sensor data analysis

Example:
In the Iris dataset, features like sepal length and petal width are continuous, hence Gaussian Naïve Bayes is appropriate.

Multinomial Naïve Bayes

Definition:
Multinomial Naïve Bayes is mainly used for discrete count-based data, especially in text classification problems.
Assumption:
Features represent frequency or count of events (e.g., word occurrences).
Mathematical Idea:
It models the probability of feature counts given a class.
Key Characteristics:

- Works on non-negative integer values
- Sensitive to feature frequency
- Often combined with TF-IDF

Applications:

- Spam email detection
- Sentiment analysis
- Document categorization

Example:
Number of times words like "offer", "free", "win" appear in an email.

Bernoulli Naïve Bayes
Definition:
Bernoulli Naïve Bayes is used when features are binary (0/1 or True/False).
Assumption:
Each feature indicates presence or absence of a characteristic.
Key Characteristics:

- Binary-valued features
- Penalizes absence of features
- Suitable for small datasets

Applications:

- Binary text classification
- Click prediction systems
- Yes/No based datasets

Example:
Whether a word appears in an email (1) or not (0).

Complement Naïve Bayes
Definition:
Complement Naïve Bayes is an improvement over Multinomial Naïve Bayes, designed to handle imbalanced datasets.
Key Characteristics:

- Uses statistics from all classes except the current class
- Reduces bias towards dominant classes

Applications:

- Large-scale document classification
- Highly skewed text datasets

Comparison of Naïve Bayes Types

Type	Data Type	Typical Use Case
Gaussian NB	Continuous	Medical, sensor, numeric data
Multinomial NB	Count-based	Text classification
Bernoulli NB	Binary	Presence/absence data
Complement NB	Imbalanced text	Large document corpora

In this experiment Multinomial Naïve Bayes is used because the dataset consists of text documents.

5. Dataset Description (Example)

Dataset Used: documents.csv

The dataset contains two document categories:

- SPORTS
- TECH

Each record contains:

- label → category of the document
- text → content of the document

```
Dataset:
    label                                  text
0  SPORTS              team won the football match
1  SPORTS           football players scored a goal
2  SPORTS               cricket team won the match
3  SPORTS             player scored a winning goal
4  SPORTS               football team won the game
5    TECH       new software update released today
6    TECH   computer software uses machine learning
7    TECH  technology company released new computer
8    TECH       machine learning improves software
9    TECH            new computer software released
```

System Requirements

Hardware

- PC/Laptop with Internet

Software

- AWS Account
- Python 3.x
- AWS SageMaker Notebook Instance

6. Step-wise Procedure (AWS Implementation)

This experiment is implemented using Amazon SageMaker and JupyterLab, which provides a managed environment for machine learning development.

Step 1: AWS Account and SageMaker Environment

Steps:

1. Login to the AWS Management Console.
2. Open Amazon SageMaker.
3. Open the available JupyterLab environment.
4. Launch/open JupyterLab.

5. Create a new Python notebook using the conda_python3 kernel.

Purpose:
SageMaker provides the JupyterLab environment required to develop and execute the Naïve Bayes document classification program.

Step 2: Create and Load the Dataset
Create a CSV file named: documents.csv
The file contains the document labels and text. The dataset is loaded using Pandas.

```
import pandas as pd
data = pd.read_csv("documents.csv")
print("Dataset:")
print(data)
```

Step 3: Prepare Training and Testing Data
The dataset is divided into training and testing data.
The training data is used to learn the word probabilities for each class.
The test data contains unseen documents that are used to evaluate the classifier.
For this experiment:

- Training data → first 8 documents
- Test data → separate documents containing both SPORTS and TECH examples

Step 4: Train the Naïve Bayes Classifier
The classifier calculates:

- Number of documents in each class
- Word frequency in each class
- Total number of words in each class
- Vocabulary of all unique words
- Prior probability of each class

Laplace smoothing is used to avoid zero probabilities.

Step 5: Predict the Class of Test Documents

For each test document, the classifier calculates the probability score for each class.
The class with the highest probability is selected as the prediction.
For example:

football team scored a goal

is predicted as:

SPORTS

Similarly:

computer machine learning software

is predicted as:

TECH

Step 6: Model Evaluation
The predictions are compared with the actual labels.
Three evaluation metrics are calculated:

- Accuracy = Correct Predictions / Total Predictions

- Precision = TP / (TP + FP)

Precision indicates how many documents predicted as a particular class actually belong to that class.

- Recall = TP / (TP + FN)

Recall indicates how many actual documents belonging to a class were correctly identified.

Step 7: Naïve Bayes Program

```
import pandas as pd
import math
from collections import Counter
data = pd.read_csv("documents.csv")
print("Dataset:")
print(data)
train_data = data.iloc[:8]
test_data = pd.DataFrame({
    "label": ["SPORTS", "SPORTS", "TECH", "TECH"],
    "text": ["football team scored a goal", "cricket team won the match", "new software update released", "computer machine learning
software"]
})
classes = train_data["label"].unique()
class_word_counts = {}
class_total_words = {}
class_doc_counts = {}
vocabulary = set()
for cls in classes:
    documents = train_data[train_data["label"] == cls]["text"]
    class_doc_counts[cls] = len(documents)
    class_word_counts[cls] = Counter()
    for text in documents:
        words = text.lower().split()
        vocabulary.update(words)
        class_word_counts[cls].update(words)
    class_total_words[cls] = sum(class_word_counts[cls].values())
total_documents = len(train_data)
def predict(text):
    words = text.lower().split()
    best_class = None
    best_probability = float("-inf")
    for cls in classes:
        probability = math.log(class_doc_counts[cls] / total_documents)
        denominator = class_total_words[cls] + len(vocabulary)
        for word in words:
            word_count = class_word_counts[cls].get(word, 0)
            word_probability = (word_count + 1) / denominator
            probability += math.log(word_probability)
        if probability > best_probability:
            best_probability = probability
            best_class = cls
    return best_class
actual = []
predicted = []
print("\nPredictions:")
for _, row in test_data.iterrows():
    prediction = predict(row["text"])
    actual.append(row["label"])
    predicted.append(prediction)
    print("Document:", row["text"])
    print("Actual:", row["label"])
    print("Predicted:", prediction)
```

```
    print()
correct = sum(a == p for a, p in zip(actual, predicted))
accuracy = correct / len(actual)
print("Accuracy:", round(accuracy * 100, 2), "%")
print("\nPrecision and Recall:")
for cls in classes:
    TP = sum(1 for a, p in zip(actual, predicted) if a == cls and p == cls)
    FP = sum(1 for a, p in zip(actual, predicted) if a != cls and p == cls)
    FN = sum(1 for a, p in zip(actual, predicted) if a == cls and p != cls)
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0
    print(cls, "-> Precision:", round(precision * 100, 2), "%", "| Recall:", round(recall * 100, 2), "%")
```

Output

The program successfully classified the test documents.

Predictions

```
Predictions:
Document: football team scored a goal
Actual: SPORTS
Predicted: SPORTS

Document: cricket team won the match
Actual: SPORTS
Predicted: SPORTS

Document: new software update released
Actual: TECH
Predicted: TECH

Document: computer machine learning software
Actual: TECH
Predicted: TECH
```

Accuracy

```
Accuracy: 100.0 %
```

Precision and Recall

```
Precision and Recall:
SPORTS -> Precision: 100.0 % | Recall: 100.0 %
TECH -> Precision: 100.0 % | Recall: 100.0 %
```

Result

The Naïve Bayes Classifier was successfully implemented for document classification using Amazon SageMaker and JupyterLab.

The classifier successfully classified the test documents into SPORTS and TECH categories.
For the given test dataset, the classifier achieved:

- Accuracy: 100%
- SPORTS Precision: 100%
- SPORTS Recall: 100%
- TECH Precision: 100%
- TECH Recall: 100%

Thus, the Naïve Bayesian Classifier successfully classified the given documents and evaluated the classification performance using accuracy, precision, and recall.

EXPERIMENT 7 - BAYESIAN NETWORK

Aim: Write a program to construct a Bayesian network considering medical data and use this model to demonstrate the diagnosis of heart patients using Heart Disease Dataset.

Learning Outcomes

After completing this experiment, students will be able to:

- Understand the concept of Bayesian Networks and their application in medical diagnosis.
- Construct a Bayesian Network using medical/heart disease data.
- Apply probabilistic inference to diagnose heart disease.
- Use Python ML libraries such as pgmpy for Bayesian Network implementation.
- Evaluate the model using accuracy, precision, and recall.
- Analyze and interpret the results obtained from the Bayesian Network model.

AWS Services Used

AWS Service	Purpose
Amazon S3	Store dataset
Amazon SageMaker	Model training and evaluation
AWS IAM	Access control
Amazon CloudWatch	Monitor training logs

Theory

A Bayesian Network is a probabilistic graphical model that represents relationships between variables using a directed acyclic graph (DAG). It uses conditional probability distributions to perform probabilistic inference.

For this practical, the Heart Disease Dataset is used. The dataset contains medical attributes such as age, sex, chest pain type, resting blood pressure, cholesterol, maximum heart rate, exercise-induced angina, and other clinical parameters. The target attribute represents whether heart disease is present or absent.

The Bayesian Network is constructed with the medical attributes as parent nodes of the target node. Since a discrete Bayesian Network is used, continuous attributes are converted into discrete categories before training.

Algorithm

1. Load the Heart Disease Dataset from the CSV file.
2. Display the dataset and its dimensions.
3. Separate the medical attributes and target variable.
4. Discretize continuous attributes.
5. Divide the dataset into training and testing data.
6. Construct the Bayesian Network.
7. Train the network using Maximum Likelihood Estimation.
8. Perform inference using Variable Elimination.
9. Predict the heart disease status for test patients.
10. Calculate accuracy, precision, and recall.
11. Use the trained network to diagnose an individual patient.

Advantages and Limitations

Advantages:

- Handles uncertainty.
- Useful for diagnosis.

- Represents variable relationships.
- Supports probabilistic inference.

Limitations:

- Needs quality data.
- Structure affects accuracy.
- Can be computationally complex.
- Discretization may lose information.

1. Dataset Description (Example)

- **Dataset Used::** Heart Disease Dataset
- **Records:** 1025
- **Attributes:** 14
- **Target:** Heart disease (0 = Absent, 1 = Present)
- **Features:** Age, sex, chest pain, blood pressure, cholesterol, heart rate, etc.
- Used to **diagnose heart disease using a Bayesian Network**.

```
Heart Disease Dataset:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

Dataset Shape: (1025, 14)

Column Names:
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
```

System Requirements

Hardware

- PC/Laptop with Internet

Software

- AWS Account
- Python 3.x
- AWS SageMaker Notebook Instance

2. Step-wise Procedure (AWS Implementation)

This experiment is implemented using Amazon SageMaker and JupyterLab, which provides a managed environment for machine learning development.

Step 1: AWS Account and SageMaker Environment

Steps:

1. Login to the AWS Management Console.
2. Open Amazon SageMaker.
3. Open the available JupyterLab environment.
4. Launch/open JupyterLab.
5. Create a new Python notebook using the conda_python3 kernel.

Purpose:
SageMaker provides the JupyterLab environment required to develop and execute the BAYESIAN NETWORK program.

Step 2: Load the Dataset
Upload the CSV file named: heart.csv
The dataset contains medical attributes and the target variable for heart disease diagnosis. The dataset is loaded using Pandas.

```
!pip install pgmpy scikit-learn pandas
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination
data = pd.read_csv("heart.csv")
print("Heart Disease Dataset:")
print(data.head())
print("\nDataset Shape:", data.shape)
print("\nColumn Names:")
print(data.columns.tolist())
```

Step 3: Prepare the Dataset
The continuous medical attributes are converted into discrete categories because a Discrete Bayesian Network is used. The following attributes are discretized:

- Age
- Resting blood pressure
- Cholesterol
- Maximum heart rate
- Oldpeak

```
continuous_columns = ["age", "trestbps", "chol", "thalach", "oldpeak"]
for column in continuous_columns:
    data[column] = pd.qcut(data[column], q=3, labels=False, duplicates="drop")
data = data.astype(int)
print("Processed Dataset:")
print(data.head())
```

```
Processed Dataset:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0    1    1   0         1     0    0        1        2      0        1      2
1    1    1   0         2     0    1        0        1      1        2      0
2    2    1   0         2     0    0        1        0      1        2      0
3    2    1   0         2     0    0        1        1      0        0      2
4    2    0   0         1     2    1        1        0      0        2      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

Processed Dataset Shape: (1025, 14)
```

Step 4: Prepare Training and Testing Data
The dataset is divided into training and testing data.

- Training data → used to train the Bayesian Network.
- Testing data → used to evaluate the model.

```
from sklearn.model_selection import train_test_split
train_data, test_data = train_test_split(data, test_size=0.2, random_state=42, stratify=data["target"])
print("Training Data:", train_data.shape)
print("Testing Data:", test_data.shape)
```

```
Training Data: (820, 14)
Testing Data: (205, 14)
```

Step 5: Construct the Bayesian Network

A Bayesian Network is constructed by defining relationships between the medical attributes and the target variable.

```
from pgmpy.models import DiscreteBayesianNetwork
model = DiscreteBayesianNetwork([
    ("age", "target"),
    ("sex", "target"),
    ("cp", "target"),
    ("trestbps", "target"),
    ("chol", "target"),
    ("fbs", "target"),
    ("restecg", "target"),
    ("thalach", "target"),
    ("exang", "target"),
    ("oldpeak", "target"),
    ("slope", "target"),
    ("ca", "target"),
    ("thal", "target")
])
print("Bayesian Network Structure:")
print(model.edges())
```

```
Bayesian Network Structure:
[('age', 'target'), ('sex', 'target'), ('cp', 'target'), ('trestbps', 'target'), ('chol', 'target'), ('fbs', 'target'), ('restecg', 'target'), ('thalach', 'target'), ('exang', 'target'
```

Step 6: Train the Bayesian Network

The Bayesian Network is trained using Maximum Likelihood Estimation.

```
from pgmpy.estimators import MaximumLikelihoodEstimator
estimator = MaximumLikelihoodEstimator(model, train_data)
cpds = estimator.get_parameters()
model.add_cpds(*cpds)
print("Bayesian Network trained successfully.")
print("Number of CPDs:", len(model.get_cpds()))
print("Model Check:", model.check_model())
```

Step 7: Perform Bayesian Inference

Variable Elimination is used to predict the heart disease status of test patients.

```
from pgmpy.inference import VariableElimination
inference = VariableElimination(model)
print("Starting diagnosis...")
predictions = []
for _, row in test_data.iterrows():
    evidence = row.drop("target").to_dict()
    result = inference.map_query(variables=["target"], evidence=evidence, show_progress=False )
    predictions.append(result["target"])
print("Diagnosis completed!")
```

```
print("Actual Values:")
print(test_data["target"].values)
print("\nPredicted Values:")
print(predictions)
```

```
Starting diagnosis...
Diagnosis completed!

Actual Values:
[0 1 0 1 0 0 1 0 1 1 0 1 0 0 1 0 0 0 0 1 0 0 1 1 0 1 0 0 1 0 1 1 0 1 0 0 1
 0 0 1 0 1 1 1 0 0 1 1 0 0 0 0 1 0 1 0 1 1 0 0 1 1 1 0 1 0 0 1 1 1 1 0 1 1
 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 0 0 1 1
 0 1 1 1 0 1 1 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 1 0 0 0 0 1 1
 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 1 1 0 0 1 1 0 0 0 1 1 1 1 0 1 0 1 1 0 1 1 0
 1 0 0 0 1 0 1 1 0 1 0 1 1 0 1 0 0 0 1 1]

Predicted Values:
[np.int64(0), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int
```

Step 8: Calculate Accuracy, Precision and Recall

The predicted results are compared with the actual values.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score
accuracy = accuracy_score(test_data["target"], predictions)
precision = precision_score(test_data["target"], predictions, zero_division=0)
recall = recall_score( test_data["target"], predictions, zero_division=0)
print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall:", round(recall * 100, 2), "%")
```

```
Accuracy: 98.54 %
Precision: 100.0 %
Recall: 97.14 %
```

Step 9: Diagnose an Individual Patient

The trained Bayesian Network is used to demonstrate the diagnosis of an individual patient.

```
patient = test_data[test_data["target"] == 1].drop(
    columns=["target"]
).iloc[0].to_dict()
diagnosis = inference.map_query(variables=["target"], evidence=patient, show_progress=False)
print("Patient Medical Data:")
print(patient)
print("\nPredicted Diagnosis:", diagnosis["target"])
if diagnosis["target"] == 1:
    print("Diagnosis: Heart Disease Present")
else:
    print("Diagnosis: Heart Disease Absent")
```

```
Patient Medical Data:
{'age': 2, 'sex': 1, 'cp': 0, 'trestbps': 0, 'chol': 1, 'fbs': 0, 'restecg': 0, 'thalach': 0, 'exang': 1, 'oldpeak': 2, 'slope': 0, 'ca': 1, 'thal': 2}

Predicted Diagnosis: 0
Diagnosis: Heart Disease Absent
```

Result

The Bayesian Network successfully classified heart disease cases using the Heart Disease Dataset. The model achieved 98.54% accuracy, 100% precision, and 97.14% recall on the test data.

EXPERIMENT 8 - EM Vs K-Means algorithm

Aim: Write a program to apply EM (Expectation-Maximization) algorithm to cluster a set of data stored in a .csv file and use the same dataset for clustering using K-Means algorithm and compare the results of these two algorithms.

Learning Outcomes

After completing this experiment, students will be able to:

- Understand EM and K-Means clustering.
- Apply clustering algorithms to a CSV dataset.
- Compare the clustering results.
- Evaluate clustering quality using Silhouette Score.

AWS Services Used

AWS Service	Purpose
Amazon S3	Store dataset
Amazon SageMaker	Model training and evaluation
AWS IAM	Access control
Amazon CloudWatch	Monitor training logs

Theory

EM (Expectation-Maximization) is an iterative clustering algorithm used to find groups in unlabeled data. It estimates the parameters of probability distributions and assigns data points to clusters based on their probabilities.

K-Means is an unsupervised clustering algorithm that divides data into a predefined number of clusters. It assigns each data point to the nearest cluster centroid and repeatedly updates the centroids until the clusters stabilize.

For this practical, the Iris Dataset is used. The same dataset is clustered using both EM and K-Means algorithms, and their results are compared using the Silhouette Score.

Advantages and Limitations

- **Advantages of EM**

- Handles probabilistic clustering.
- Works well with overlapping clusters.
- Estimates cluster distributions.
- Provides flexible clustering.

- **Limitations of EM**

- Can be computationally expensive.
- May converge to a local optimum.
- Sensitive to initialization.
- Requires the number of clusters.

- **Advantages of K-Means**

- Simple and easy to implement.
- Fast for large datasets.
- Easy to understand.
- Efficient for well-separated clusters.

- **Limitations of K-Means**

- Requires number of clusters in advance.
- Sensitive to initial centroids.
- Sensitive to outliers.
- Works best with spherical clusters.

1. Dataset Description (Example)

- **Dataset Used:** Iris Dataset
- **Records**: 150
- **Attributes:** 5
- **Features:** Sepal length, sepal width, petal length, petal width
- **Species:** Setosa, Versicolor, Virginica
- **Clusters:** 3
- Used to compare clustering results using EM and K-Means algorithms.

```
Dataset:
     sepal_length  sepal_width  petal_length  petal_width         species
0             5.1          3.5           1.4          0.2     Iris-setosa
1             4.9          3.0           1.4          0.2     Iris-setosa
2             4.7          3.2           1.3          0.2     Iris-setosa
3             4.6          3.1           1.5          0.2     Iris-setosa
4             5.0          3.6           1.4          0.2     Iris-setosa
..            ...          ...           ...          ...             ...
145           6.7          3.0           5.2          2.3  Iris-virginica
146           6.3          2.5           5.0          1.9  Iris-virginica
147           6.5          3.0           5.2          2.0  Iris-virginica
148           6.2          3.4           5.4          2.3  Iris-virginica
149           5.9          3.0           5.1          1.8  Iris-virginica

[150 rows x 5 columns]
```

System Requirements

Hardware

- PC/Laptop with Internet

Software

- AWS Account
- Python 3.x
- AWS SageMaker Notebook Instance

2. Step-wise Procedure (AWS Implementation)

This experiment is implemented using Amazon SageMaker and JupyterLab, which provides a managed environment for machine learning development.

Step 1: AWS Account and SageMaker Environment

Steps:

1. Login to the AWS Management Console.
2. Open Amazon SageMaker.
3. Open the available JupyterLab environment.
4. Launch/open JupyterLab.
5. Create a new Python notebook using the conda_python3 kernel.

Purpose:

SageMaker provides the JupyterLab environment required to develop and execute the EM Algorithm and K-Means Algorithm program.

Step 2: Load the Dataset
Upload the CSV file named: iris.csv
The dataset contains four numerical features and the species column. The dataset is loaded using Pandas.

```
import pandas as pd
data = pd.read_csv("iris.csv")
print("Dataset:")
print(data)
print("\nDataset Shape:", data.shape)
print("\nColumn Names:")
print(data.columns.tolist())
```

```
Dataset:
     sepal_length  sepal_width  petal_length  petal_width         species
0             5.1          3.5           1.4          0.2     Iris-setosa
1             4.9          3.0           1.4          0.2     Iris-setosa
2             4.7          3.2           1.3          0.2     Iris-setosa
3             4.6          3.1           1.5          0.2     Iris-setosa
4             5.0          3.6           1.4          0.2     Iris-setosa
..            ...          ...           ...          ...             ...
145           6.7          3.0           5.2          2.3  Iris-virginica
146           6.3          2.5           5.0          1.9  Iris-virginica
147           6.5          3.0           5.2          2.0  Iris-virginica
148           6.2          3.4           5.4          2.3  Iris-virginica
149           5.9          3.0           5.1          1.8  Iris-virginica

[150 rows x 5 columns]

Dataset Shape: (150, 5)

Column Names:
['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
```

Step 3: Prepare the Dataset
The species column is not used for clustering because EM and K-Means are unsupervised learning algorithms. The four numerical features are selected and standardized before clustering.

```
from sklearn.preprocessing import StandardScaler
X = data.drop("species", axis=1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Data used for clustering:")
print(X.head())
print("\nScaled data shape:", X_scaled.shape)
```

```
Data used for clustering:
   sepal_length  sepal_width  petal_length  petal_width
0           5.1          3.5           1.4          0.2
1           4.9          3.0           1.4          0.2
2           4.7          3.2           1.3          0.2
3           4.6          3.1           1.5          0.2
4           5.0          3.6           1.4          0.2

Scaled data shape: (150, 4)
```

Step 4: Apply EM Algorithm
The Gaussian Mixture Model (GMM) is used to perform clustering using the EM algorithm. The algorithm estimates the parameters of the Gaussian distributions and assigns the data points to three clusters.

```
from sklearn.mixture import GaussianMixture
em = GaussianMixture( n_components=3, random_state=42)
em_labels = em.fit_predict(X_scaled)
print("EM Clustering Results:")
print(em_labels)
```

```
EM Clustering Results:
[1 1 1 2 1 1 1 1 2 1 1 1 1 2 1 1 1 1 1 1 1 1 2 1 1 1 1 2 1 1 1 1 1 1 1
 1 2 1 1 2 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```

Step 5: Apply K-Means Algorithm

The same dataset is used for clustering using the K-Means algorithm.

```
from sklearn.cluster import KMeans
kmeans = KMeans( n_clusters=3, random_state=42, n_init=10 )
kmeans_labels = kmeans.fit_predict(X_scaled)
print("K-Means Clustering Results:")
print(kmeans_labels)
```

```
K-Means Clustering Results:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 0 0 0 2 0 0 0 0 0 0 0 0 2 0 0 0 0 2 0 0 0
 0 2 2 2 0 0 0 0 0 0 0 2 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 2 2 2 0 2 2 2 2
 2 2 0 0 2 2 2 2 0 2 0 2 0 2 2 0 2 2 2 2 2 2 0 0 2 2 2 0 2 2 2 0 2 2 2 0 2
 2 0]
```

Step 6: Evaluate Clustering Quality

The Silhouette Score is used to evaluate the quality of clusters produced by EM and K-Means. A higher Silhouette Score indicates better-defined clusters.

```
from sklearn.metrics import silhouette_score
em_score = silhouette_score(X_scaled, em_labels)
kmeans_score = silhouette_score(X_scaled, kmeans_labels)
print("Clustering Quality:")
print("EM Silhouette Score:", round(em_score, 4))
print("K-Means Silhouette Score:", round(kmeans_score, 4))
if em_score > kmeans_score:
    print("\nEM produced better quality clusters.")
elif kmeans_score > em_score:
    print("\nK-Means produced better quality clusters.")
else:
    print("\nBoth algorithms produced similar quality clusters.")
```

```
Clustering Quality:
EM Silhouette Score: 0.4092
K-Means Silhouette Score: 0.459

K-Means produced better quality clusters.
```

Step 7: Compare Cluster Distribution
The number of data points assigned to each cluster is displayed for both algorithms.

```
import numpy as np
print("EM Cluster Distribution:")
print(np.bincount(em_labels))
print("\nK-Means Cluster Distribution:")
print(np.bincount(kmeans_labels))
```

```
EM Cluster Distribution:
[100  43   7]

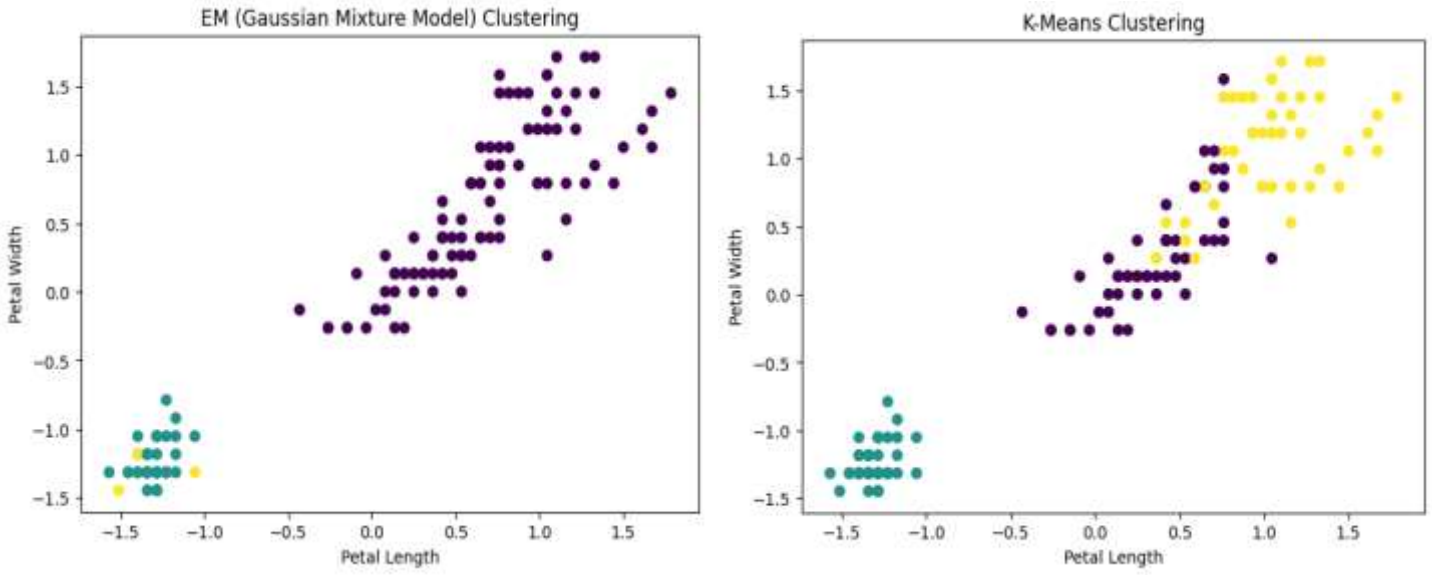
K-Means Cluster Distribution:
[53 50 47]
```

Step 8: EM Clustering Graph
The EM clustering results are visualized using a scatter plot.

```
!pip install matplotlib
import matplotlib.pyplot as plt
plt.figure(figsize=(7,5))
plt.scatter(X_scaled[:,2], X_scaled[:,3], c=em_labels)
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("EM (Gaussian Mixture Model) Clustering")
plt.show()
```

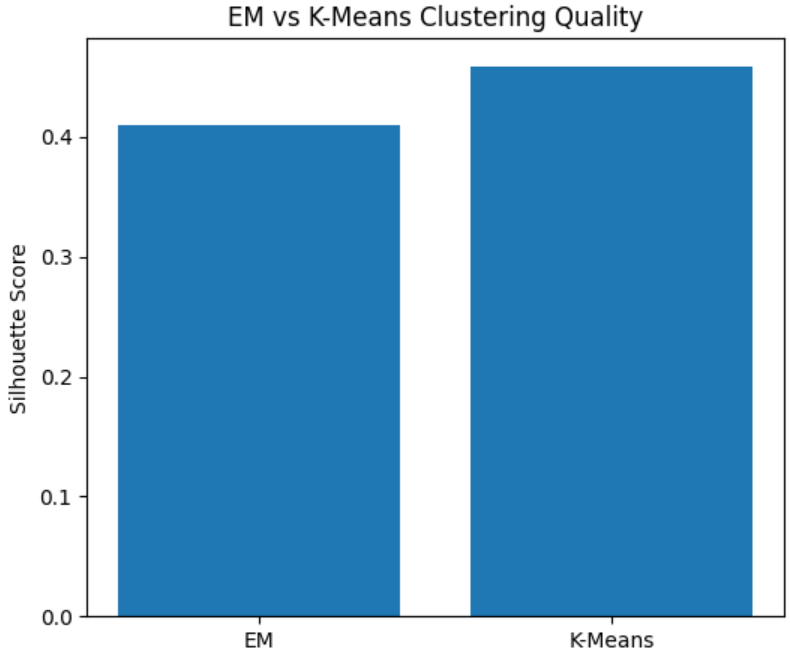
Step 9: K-Means Clustering Graph
The K-Means clustering results are visualized using a scatter plot.

```
plt.figure(figsize=(7,5))
plt.scatter(X_scaled[:,2], X_scaled[:,3], c=kmeans_labels)
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("K-Means Clustering")
plt.show()
```



Step 10: EM vs K-Means Comparison Graph
The Silhouette Scores of both algorithms are compared using a bar graph.

```
plt.figure(figsize=(6,5))
plt.bar(["EM", "K-Means"], [em_score, kmeans_score])
plt.ylabel("Silhouette Score")
plt.title("EM vs K-Means Clustering Quality")
plt.show()
```



Result
The EM and K-Means clustering algorithms were successfully applied to the Iris Dataset. The EM algorithm produced clusters with a Silhouette Score of 0.4092, while the K-Means algorithm produced a higher Silhouette Score of 0.4590.
The EM algorithm generated cluster sizes of 100, 43, and 7, whereas K-Means generated more balanced clusters of 53, 50, and 47.
Since K-Means achieved the higher Silhouette Score of 0.4590, it produced better quality clustering results than EM for this dataset.

EXPERIMENT 9 - Non-Parametric Locally Weighted Regression Algorithm

Aim: Write a program to implement the non-parametric Locally Weighted Regression algorithm in order to fit data points.

Learning Outcomes

After completing this experiment, students will be able to:

- Understand the concept of non-parametric Locally Weighted Regression (LWR).
- Implement Locally Weighted Regression using Python.
- Understand the effect of the bandwidth parameter on the regression curve.
- Fit a regression model locally around each query point.
- Visualize the fitted regression curve using graphs.
- Perform machine learning experiments using Amazon SageMaker and JupyterLab.

AWS Services Used

AWS Service	Purpose
Amazon S3	Store dataset
Amazon SageMaker	Model training and evaluation
AWS IAM	Access control
Amazon CloudWatch	Monitor training logs

Theory

Locally Weighted Regression (LWR) is a non-parametric regression algorithm used to fit data points. Unlike ordinary linear regression, which learns one global model for the complete dataset, LWR gives different weights to training data points depending on their distance from the query point.

Data points that are closer to the query point receive higher weights, while distant points receive lower weights. A Gaussian kernel is commonly used to calculate the weights.

The weight is calculated using:

$$w_i = e^{\frac{-((x_i - x)^2)}{2T^2}}$$

where:

- x_i = training data point
- x = query point
- w_i = weight of the training data point
- tau = bandwidth parameter

A smaller value of T gives more importance to nearby points and produces a more local fit. A larger value of T produces a smoother regression curve.

For this experiment, the Salary Dataset is used. The relationship between Years of Experience and Salary is fitted using Locally Weighted Regression and visualized using graphs.

Advantages and Limitations

- **Advantages**

- Simple and easy to understand.
- Can model non-linear relationships.
- Gives more importance to nearby data points.
- Does not require a fixed global model.
- Useful for locally fitting data.

- **Limitations**

- Computationally expensive for large datasets.
- Sensitive to the choice of bandwidth parameter.
- Requires calculations for each query point.
- Performance can decrease for high-dimensional data..

1. Dataset Description (Example)

- **Dataset Used:** Salary Dataset for Simple Linear Regression
- **Records**: 30
- **Attributes:** 2
- **Input Feature:** Years of Experience
- **Target:** Salary

The dataset contains information about the number of years of experience and the corresponding salary. It is used to demonstrate how Locally Weighted Regression fits a regression curve to the given data points.

System Requirements

Hardware

- PC/Laptop with Internet

Software

- AWS Account
- Python 3.x
- AWS SageMaker Notebook Instance

2. Step-wise Procedure (AWS Implementation)

This experiment is implemented using Amazon SageMaker and JupyterLab, which provides a managed environment for machine learning development.

Step 1: AWS Account and SageMaker Environment

Steps:

1. Login to the AWS Management Console.
2. Open Amazon SageMaker.
3. Open the available JupyterLab environment.
4. Launch/open JupyterLab.
5. Create a new Python notebook using the conda_python3 kernel.

Purpose:

SageMaker provides the JupyterLab environment required to develop and execute the Locally Weighted Regression program.

Step 2: Load the Dataset

Upload the CSV file named: salarty_data.csv

The dataset contains two columns: YearsExperience and Salary. The dataset is loaded using Pandas.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data = pd.read_csv("Salary_Data.csv")
print("Dataset:")
print(data.head())
print("\nDataset Shape:", data.shape)
print("\nColumn Names:")
print(data.columns.tolist())
```

```
Dataset:
   YearsExperience   Salary
0              1.1  39343.0
1              1.3  46205.0
2              1.5  37731.0
3              2.0  43525.0
4              2.2  39891.0

Dataset Shape: (30, 2)

Column Names:
['YearsExperience', 'Salary']
```

Step 3: Prepare the Dataset

The YearsExperience column is used as the input feature and Salary is used as the target variable.

```
X = data["YearsExperience"].values
y = data["Salary"].values
print("Years of Experience:")
print(X)
print("\nSalary:")
print(y)
```

```
Experience:
[ 1.1  1.3  1.5  2.   2.2  2.9  3.   3.2  3.2  3.7  3.9  4.   4.   4.1
  4.5  4.9  5.1  5.3  5.9  6.   6.8  7.1  7.9  8.2  8.7  9.   9.5  9.6
 10.3 10.5]

Salary:
[ 39343.  46205.  37731.  43525.  39891.  56642.  60150.  54445.  64445.
  57189.  63218.  55794.  56957.  57081.  61111.  67938.  66029.  83088.
  81363.  93940.  91738.  98273. 101302. 113812. 109431. 105582. 116969.
 112635. 122391. 121872.]
```

Step 4: Implement Locally Weighted Regression

The Locally Weighted Regression function is implemented using a Gaussian kernel. The weight of each data point is calculated according to its distance from the query point.

```
def locally_weighted_regression(x_query, X, y, tau):
    X_matrix = np.c_[np.ones(len(X)), X]
    x_query_matrix = np.array([1, x_query])
    weights = np.exp(-((X - x_query) ** 2) / (2 * tau ** 2))
    W = np.diag(weights)
    theta = np.linalg.pinv(X_matrix.T @ W @ X_matrix) @ (X_matrix.T @ W @ y)
    return x_query_matrix @ theta
```

Step 5: Generate LWR Predictions

A bandwidth value of 1.0 is selected and predictions are generated for a range of experience values.

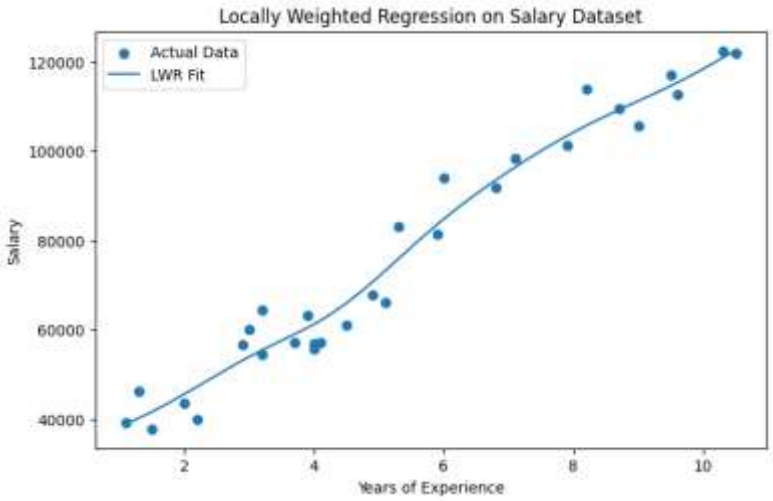
```
tau = 1.0
X_test = np.linspace(X.min(), X.max(), 200)
y_pred = np.array([ locally_weighted_regression(x, X, y, tau) for x in X_test])
print("LWR predictions generated successfully.")
```

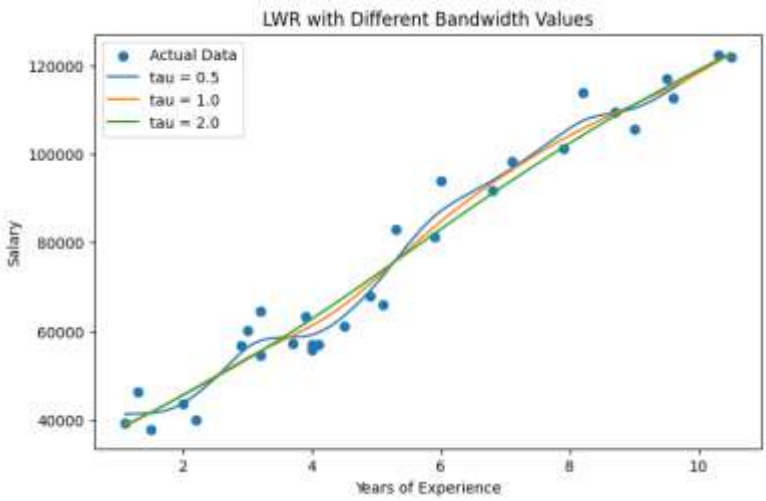
```
LWR predictions generated successfully.
```

Step 6: Draw LWR Graph

The original salary data points and the fitted Locally Weighted Regression curve are plotted.

```
plt.figure(figsize=(8,5))
plt.scatter(X, y, label="Actual Data")
plt.plot(X_test, y_pred, label="LWR Fit")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Locally Weighted Regression on Salary Dataset")
plt.legend()
plt.show()
```





Step 7: Effect of Different Bandwidth Values

Different values of tau are used to observe their effect on the regression curve. The smaller bandwidth produces a more local curve, while the larger bandwidth produces a smoother curve.

```
plt.figure(figsize=(8,5))
plt.scatter(X, y, label="Actual Data")
for tau in [0.5, 1.0, 2.0]:
    y_pred = np.array([ locally_weighted_regression(x, X, y, tau) for x in X_test])
    plt.plot(X_test, y_pred, label=f"tau = {tau}")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("LWR with Different Bandwidth Values")
plt.legend()
plt.show()
```

Result

The non-parametric Locally Weighted Regression algorithm was successfully implemented using the Kaggle Salary Dataset in Amazon SageMaker.

The algorithm fitted a regression curve by assigning higher weights to data points closer to each query point. The relationship between Years of Experience and Salary was visualized using a scatter plot and the LWR fitted curve. The effect of different bandwidth values was also observed. A smaller bandwidth produced a more local fit, while a larger bandwidth produced a smoother regression curve.

EXPERIMENT 10 – Amazon SageMaker (Labs 3.1 – 3.6)

Aim: Write a program to

- **Lab 3.1 - Amazon SageMaker - Creating and importing data**
- **Lab 3.2 - Exploring Data**
- **Lab 3.3 - Encoding Categorical Data**
- **Lab 3.4 - Training a model**
- **Lab 3.5 - Generating model performance metrics**
- **Lab 3.6 - Hyperparameter Tuning**

Learning Outcomes

After completing this experiment, students will be able to:

- Understand the process of creating and importing datasets using Amazon SageMaker and JupyterLab.
- Explore datasets using Python and Pandas to understand their structure, features, and statistical properties.
- Encode categorical data into numerical values for machine learning applications.
- Train a Decision Tree classification model using the Iris dataset.
- Evaluate model performance using accuracy, precision, recall, confusion matrix, and classification report.
- Perform hyperparameter tuning using GridSearchCV to identify suitable model parameters.
- Understand the use of Amazon SageMaker and JupyterLab for machine learning experiments.

AWS Services Used

AWS Service	Purpose
Amazon S3	Store dataset
Amazon SageMaker	Model training and evaluation
AWS IAM	Access control
Amazon CloudWatch	Monitor training logs

Theory

Locally Amazon SageMaker is a cloud-based machine learning service provided by AWS that helps users build, train, and deploy machine learning models. JupyterLab provides an interactive environment to execute Python code and perform data analysis.

The Iris dataset contains flower measurements and their corresponding species. In this experiment, the dataset is imported into JupyterLab, explored using Pandas, and prepared for machine learning.

Categorical encoding converts text-based categories into numerical values. A Decision Tree classifier is then trained to predict flower species based on their measurements.

Model performance is evaluated using accuracy, precision, recall, and a confusion matrix. Hyperparameter tuning using GridSearchCV helps identify suitable model settings through cross-validation.

1. Dataset Description (Example)

- **Dataset Used:** Iris Dataset for Classification
- **Records:** 150
- **Attributes:** 5 (4 input features and 1 target attribute)
- **Input Features:** Sepal Length, Sepal Width, Petal Length, Petal Width
- **Target:** Species

The Iris dataset contains measurements of iris flowers and their corresponding species. It includes three species: Iris-setosa, Iris-versicolor, and Iris-virginica. It is used to demonstrate data importing, exploration, categorical

encoding, Decision Tree model training, performance evaluation, and hyperparameter tuning using Python in Amazon SageMaker JupyterLab.

System Requirements
Hardware

- PC/Laptop with Internet

Software

- AWS SageMaker Notebook Instance
- AWS Academy Account
- Python 3.x
- Pandas
- Scikit-learn

2. Step-wise Procedure (AWS Implementation)
This experiment is implemented using Amazon SageMaker and JupyterLab, which provides a managed environment for machine learning development.

Step 1: AWS Account and SageMaker Environment
Steps:

1. Login to the AWS Management Console.
2. Open Amazon SageMaker AI.
3. Open the available JupyterLab environment.
4. Launch/open JupyterLab.
5. Create a new Python notebook using the conda_python3 kernel.

Purpose:
Amazon SageMaker and JupyterLab provide the environment required to import datasets, explore data, train machine learning models, and evaluate their performance.

Step 2: Load the Dataset
Upload the CSV file named Iris.csv to the JupyterLab environment.
The dataset contains flower measurements and species labels.

```
df = pd.read_csv("Iris.csv") #3.1
print("Dataset imported successfully!")
print("Dataset shape:", df.shape)
df.head()
```

```
Dataset imported successfully!
Dataset shape: (150, 6)
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Step 3: Explore the Dataset

```
print("Column names:") #3.2
print(df.columns.tolist())
print("\nDataset information:")
df.info()
```

```
print("\nMissing values:")
print(df.isnull().sum())
print("Dataset shape:", df.shape)
print("Statistical Summary:")
display(df.describe())
print("\nSpecies Count:")
print(df["Species"].value_counts())
```

```
Column names:
['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
Missing values:
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
Dataset shape: (150, 6)
Statistical Summary:
```

```
Species Count:
Species
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

Statistical Summary:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

Step 4: Encode Categorical Data

The Species column contains categorical labels. Label Encoding converts these labels into numerical values.

```
%pip install scikit-learn #3.3
from sklearn.preprocessing import LabelEncoder
print("Original Species values:")
print(df["Species"].unique())
le = LabelEncoder()
df["Species_Encoded"] = le.fit_transform(df["Species"])
print("\nEncoded dataset:")
display(df.head())
print("\nEncoding mapping:")
for i, species in enumerate(le.classes_):
    print(species, "=", i)
df.to_csv("Iris_Encoded.csv", index=False)
print("Encoded dataset saved successfully!")
```

```
Original Species values:
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']

Encoded dataset:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Species_Encoded
0	1	5.1	3.5	1.4	0.2	Iris-setosa	0
1	2	4.9	3.0	1.4	0.2	Iris-setosa	0
2	3	4.7	3.2	1.3	0.2	Iris-setosa	0
3	4	4.6	3.1	1.5	0.2	Iris-setosa	0
4	5	5.0	3.6	1.4	0.2	Iris-setosa	0

```
Encoding mapping:
Iris-setosa = 0
Iris-versicolor = 1
Iris-virginica = 2
```

Step 5: Prepare the Dataset and Train the Model

```
import pandas as pd #3.4
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
df = pd.read_csv("Iris.csv")
df = df.loc[:, ~df.columns.str.contains("^Unnamed|^Id$", case=False)]
print(df.head())
print(df.columns.tolist())
target_col = "Species"
X = df.drop(columns=[target_col])
y = df[target_col]
X = X.select_dtypes(include="number")
print("Features:", X.columns.tolist())
print("Target:", target_col)
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y)
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
print("Model trained successfully!")
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Actual values:    ", list(y_test))
print("Predicted values: ", list(y_pred))
print("Model Accuracy: {:.2f}%".format(accuracy * 100))
```

```
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0            5.1           3.5            1.4           0.2  Iris-setosa
1            4.9           3.0            1.4           0.2  Iris-setosa
2            4.7           3.2            1.3           0.2  Iris-setosa
3            4.6           3.1            1.5           0.2  Iris-setosa
4            5.0           3.6            1.4           0.2  Iris-setosa
['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']
```

```
Features: ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
Target: Species
```

```
Training samples: 120
Testing samples: 30
```

```
Actual values:    ['Iris-setosa', 'Iris-virginica', 'Iris-versicolor', 'Iris-versicolor', 'I
Predicted values: ['Iris-setosa', 'Iris-virginica', 'Iris-versicolor', 'Iris-versicolor', 'I
Model Accuracy: 93.33%
```

Step 6: Generate Model Performance Metrics

```
#3.5
from sklearn.metrics import ( accuracy_score, precision_score, recall_score, confusion_matrix, classification_report)
y_pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
print("Precision:", round( precision_score(y_test, y_pred, average="weighted", zero_division=0) * 100, 2), "%")
print("Recall:", round( recall_score(y_test, y_pred, average="weighted", zero_division=0) * 100, 2), "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Accuracy: 93.33 %
Precision: 93.33 %
Recall: 93.33 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:
                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       0.90      0.90      0.90        10
 Iris-virginica       0.90      0.90      0.90        10

       accuracy                           0.93        30
      macro avg       0.93      0.93      0.93        30
   weighted avg       0.93      0.93      0.93        30
```

Step 7: Perform Hyperparameter Tuning

```
#3.6
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import accuracy_score
X = df.drop(columns=["Species", "Species_Encoded"], errors="ignore")
X = X.select_dtypes(include="number")
y = df["Species"]
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y)
model = DecisionTreeClassifier(random_state=42)
param_grid = {"criterion": ["gini", "entropy"],"max_depth": [2, 3, 4, 5, None], "min_samples_split": [2, 5, 10]}
grid_search = GridSearchCV( estimator=model, param_grid=param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)
print("Best Hyperparameters:", grid_search.best_params_)
print("Best Cross-Validation Accuracy:", round(grid_search.best_score_ * 100, 2), "%")
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
print("Test Accuracy:", round(accuracy_score(y_test, y_pred) * 100, 2), "%")
```

```
Best Hyperparameters: {'criterion': 'gini', 'max_depth': 4, 'min_samples_split': 2}
Best Cross-Validation Accuracy: 94.17 %
Test Accuracy: 93.33 %
```

Result

The Iris dataset was imported and explored successfully using Amazon SageMaker JupyterLab. The categorical species column was encoded, and a Decision Tree classifier was trained to predict flower species. Model performance was evaluated using accuracy, precision, recall, and a confusion matrix. Hyperparameter tuning was performed using GridSearchCV.

The model achieved 93.33% test accuracy, and the best cross-validation accuracy after hyperparameter tuning was 94.17%.